

**Station intelligente de surveillance
environnementale basée sur STM32
Recherche et réalisation de la
virtualisation**

Mémoire de recherche et développement

10 mai 2026

Résumé

Une nouvelle vague de révolution informationnelle, représentée par l'Internet des objets (IoT) et les technologies embarquées, est en train de remodeler le mode de fonctionnement de la société. La surveillance environnementale intelligente est devenue un élément clé dans la construction des villes intelligentes. Cependant, dans le processus d'enseignement et de recherche et développement des systèmes embarqués, les développeurs sont confrontés à de nombreux défis : coût élevé du matériel, processus de débogage complexes, seuil de compréhension élevé des capteurs et inconvénient de la visualisation des données. Le mode de développement embarqué traditionnel exige que les apprenants acquièrent de nombreux équipements matériels et effectuent des débogages répétés sur des circuits réels, ce qui augmente non seulement les coûts d'apprentissage, mais limite aussi la flexibilité et l'évolutivité des scénarios pédagogiques.

Pour répondre aux problèmes susmentionnés, cet article propose et met en œuvre un système de virtualisation d'une station intelligente de surveillance environnementale basée sur STM32. Les contributions principales de cette recherche se décomposent en trois aspects : premièrement, la conception et la réalisation d'une plateforme matérielle virtuelle haute fidélité du STM32F103C8T6, simulant complètement le noyau MCU, le système mémoire, les GPIO, l'ADC, l'I2C, l'UART et les timers, offrant un environnement logiciel permettant d'exécuter des programmes embarqués sans matériel réel ; deuxièmement, le développement de pilotes virtuels pour six capteurs environnementaux courants, incluant le capteur de température et d'humidité DHT11, le capteur de qualité de l'air MQ-135, le capteur de luminosité BH1750, le capteur de pression BMP280 et le capteur de bruit, en établissant un cadre de modélisation unifié des capteurs et un modèle de bruit, assurant ainsi la réalité et la fiabilité des données simulées ; enfin, la construction d'un système de visualisation Web basé sur Python Flask, réalisant l'acquisition, le stockage, l'alerte et l'affichage graphique des données des capteurs en temps réel, interagissant avec le frontend via une API RESTful et supportant le réglage à distance des paramètres environnementaux.

Les résultats expérimentaux montrent que l'efficacité d'exécution des instructions de la plateforme virtuelle STM32 est hautement cohérente avec le matériel réel, la précision de simulation des capteurs atteint ou approche les valeurs nominées des fiches techniques des capteurs réels, et le temps de réponse des API du système Web est inférieur à 100 ms, répondant aux besoins de surveillance en temps réel. Cette recherche fournit une plateforme expérimentale à faible seuil et haute flexibilité pour l'enseignement des systèmes embarqués, ainsi qu'un environnement de développement logiciel efficace pour la validation de prototypes d'applications IoT, créant une valeur positive tant sur le plan théorique que pratique.

Mots-clés : STM32 ; virtualisation ; surveillance environnementale ; simulation de cap-

teurs ; Internet des objets ; enseignement des systèmes embarqués

Table des matières

Résumé	1
1 Introduction	4
1.1 Contexte et importance de la recherche	4
1.2 État de l'art	5
1.2.1 Recherche sur les plateformes de simulation embarquée	5
1.2.2 Recherche sur les systèmes de surveillance environnementale	6
1.3 Contributions principales	7
1.4 Structure du document	8
2 Conception générale du système	9
2.1 Analyse des besoins fonctionnels	9
2.1.1 Besoins fonctionnels	9
2.1.2 Besoins non fonctionnels	10
2.2 Architecture globale	10
2.3 Choix technologiques	12
3 Conception de la plateforme matérielle virtuelle STM32	13
3.1 Conception du cœur MCU virtuel	14
3.1.1 Vue d'ensemble des paramètres du STM32F103C8T6	14
3.1.2 Simulation du timer SysTick	14
3.2 Conception du système mémoire	15
3.2.1 Modèle de cartographie mémoire	15
3.2.2 Implémentation du mode little-endian	16
3.2.3 Cartographie des registres	16
3.3 Simulation des périphériques	17
3.3.1 GPIO (entrées/sorties générales)	17
3.3.2 ADC (convertisseur analogique-numérique)	18
3.3.3 Contrôleur de bus I2C	18
3.3.4 Communication série UART	19
3.3.5 TIM (timer général)	21
4 Pilotes de capteurs et acquisition de données	23
4.1 Principes de virtualisation des capteurs	23
4.1.1 Cadre de modélisation général	23
4.1.2 Interface des pilotes de capteurs	24
4.2 Capteur de température et d'humidité DHT11	24
4.2.1 Vue d'ensemble du capteur	24
4.2.2 Simulation du protocole one-wire	25

4.3	Capteur de qualite de l'air MQ-135	26
4.3.1	Principe du capteur	26
4.3.2	Deduction tension-concentration	27
4.3.3	Implementation du pilote virtuel	27
4.4	Capteur de luminosite BH1750	28
4.4.1	Vue d'ensemble du capteur	28
4.4.2	Formule de calcul de la luminosite	28
4.5	Capteur de pression barometrique BMP280	29
4.5.1	Vue d'ensemble du capteur	29
4.5.2	Verification de l'identifiant de la puce	29
4.5.3	ADC 24 bits et coefficients de calibration	30
4.6	Capteur de bruit	30
4.6.1	Principe du capteur	30
4.6.2	Mapping lineaire tension	31
4.7	Conception de la boucle d'acquisition de donnees	32
5	Conception et realisation du logiciel	34
5.1	Module de stockage des donnees	34
5.1.1	Structure des tables de la base de donnees	34
5.1.2	Interface Python sqlite3	36
5.1.3	Operations CRUD	36
5.1.4	Requetes statistiques	37
5.2	Conception du systeme d'alarmes	37
5.2.1	Configuration des seuils d'alarme	38
5.2.2	Logique de verification des alarmes	39
5.2.3	Persistence des enregistrements d'alarme	40
5.3	Systeme de visualisation Web	41
5.3.1	Conception des routes Flask	41
5.3.2	Format de reponse JSON RESTful	42
5.3.3	Configuration du graphique a deux courbes Chart.js	43
5.3.4	Mecanisme d'interrogation a 1 seconde	44
5.4	Conception du panneau de controle	45
5.4.1	Conception du mecanisme IPC	45
5.4.2	Analyse de la commande set	46
5.4.3	Limites des plages de parametres	47
6	Tests et validation du systeme	48
6.1	Environnement de test	48
6.2	Tests fonctionnels	49
6.3	Tests de performance	50

6.3.1	Temps de reponse des API	50
6.3.2	Stabilite de la periode d'acquisition	50
6.3.3	Occupation memoire	51
6.4	Verification de la precision des capteurs	51
6.4.1	Analyse comparative de la precision	52
6.4.2	Tests de stabilite a long terme	52
6.5	Validation de la fonction d'alarme	53
6.5.1	Scenario d'alarme haute temperature	53
6.5.2	Scenario d'alarme basse pression	53
6.5.3	Scenario d'alarmes simultanees multiples	54
6.6	Presentation de l'interface Web	54
7	Conclusion et perspectives	55
7.1	Synthèse des travaux	55
7.2	Perspectives de travaux futurs	57
	References	59

1 Introduction

1.1 Contexte et importance de la recherche

À l'heure actuelle, une nouvelle vague de révolution informationnelle, représentée par l'Internet des objets (Internet of Things, IoT), est en train de remodeler le mode de fonctionnement de la société. La réalisation des applications telles que les villes intelligentes, l'automatisation industrielle, l'agriculture de précision et les maisons intelligentes repose sur la surveillance continue des paramètres physiques de l'environnement et l'analyse intelligente de ces données. Les indicateurs de température, d'humidité, de qualité de l'air, d'intensité lumineuse, de pression atmosphérique et de niveau sonore concernent non seulement la santé et le confort des individus, mais influencent également l'efficacité énergétique dans les contextes industriels. Par conséquent, la mise au point de solutions de surveillance environnementale intelligentes, stables, à coût contrôlé et déployables de manière flexible présente une valeur significative tant sur le plan des services sociaux que de l'économie.

Les systèmes de calcul embarqués constituent la base technologique des nœuds périphériques de l'IoT, responsables de l'acquisition, du prétraitement et de la transmission des données des capteurs. La famille de produits STM32 lancée par STMicroelectronics, et en particulier le modèle STM32F103C8T6 équipé d'un cœur Cortex-M3, est largement adopté dans l'enseignement universitaire et la pratique industrielle en raison de son bon compromis entre performance de calcul et consommation d'énergie, de la diversité des ressources périphériques intégrées et de la maturité de la chaîne d'outils associée. Cette puce a accumulé une riche expérience d'application dans des domaines tels que la surveillance environnementale, le contrôle industriel et l'électronique grand public.

Cependant, que ce soit dans un contexte éducatif ou pour l'auto-apprentissage, l'initiation et l'avancement dans les systèmes embarqués se heurtent à plusieurs obstacles concrets :

(1) **Coût élevé du matériel.** Un système de surveillance environnemental complet nécessite l'achat d'une carte de développement STM32, de divers modules de capteurs, d'un débogueur, d'une alimentation et de câbles de connexion, pour un coût unitaire généralement compris entre plusieurs centaines et plus d'un millier de yuans. Pour les laboratoires universitaires, équiper des dizaines de postes de travail complets représente un investissement financier considérable ; pour les apprenants individuels, le coût d'entrée élevé constitue également une barrière.

(2) **Processus de débogage complexe.** Le débogage des systèmes embarqués dépend de l'état réel des connexions des circuits matériels. Un simple câble desserré, une résistance mal soudée ou une erreur de configuration de broche peut entraîner un dysfonctionnement du système. Face à des pannes matérielles complexes, les débutants ont souvent du mal à identifier rapidement la cause profonde, ce qui leur fait perdre beaucoup de temps en dépannage matériel au lieu de se concentrer sur l'apprentissage des algorithmes logiciels

et de la logique système.

(3) **Seuil de compréhension élevé des capteurs.** Les principes de fonctionnement, les protocoles de communication et les méthodes de traitement des données des différents capteurs varient considérablement. Par exemple, le DHT11 utilise un protocole one-wire pour transmettre 40 bits de données, le MQ-135 nécessite l'acquisition de tension via l'ADC et l'inférence de la concentration de gaz à l'aide de modèles mathématiques complexes, tandis que le BMP280 implique la communication I2C, la lecture d'un ADC 24 bits et des algorithmes de compensation thermique. La compréhension approfondie des mécanismes de fonctionnement de ces capteurs exige de la part des apprenants des connaissances en circuits analogiques, circuits numériques, protocoles de communication et traitement des données.

(4) **Inconvénient de la visualisation des données.** Dans les environnements de développement embarqué traditionnels, les données des capteurs sont généralement sorties sous forme de texte via le port série. Les développeurs doivent consulter les valeurs brutes dans un assistant de port série, manquant ainsi de moyens de visualisation intuitifs des données. Même avec l'aide de logiciels hôtes, un travail de développement supplémentaire est souvent nécessaire, et les fonctionnalités sont limitées, rendant difficile la réalisation de fonctions avancées telles que l'accès à distance via le Web, la consultation des données historiques et les alertes intelligentes.

Sur la base de l'analyse ci-dessus, le développement d'un système de virtualisation d'une station intelligente de surveillance environnementale basée sur STM32, permettant aux apprenants de réaliser l'ensemble du processus de développement embarqué sans acquérir de matériel physique, présente une double valeur : d'une part, il fournit un support expérimental à faible coût pour les programmes de cours ; d'autre part, il offre un environnement d'itération rapide pour la validation précoce des concepts de produits.

1.2 État de l'art

1.2.1 Recherche sur les plateformes de simulation embarquée

La recherche sur la virtualisation des processeurs et la simulation des systèmes embarqués a débuté tôt, tant dans le milieu académique qu'industriel. QEMU (Quick Emulator), en tant qu'émulateur de machine universel open source, est compatible avec plusieurs architectures de jeux d'instructions, y compris la série ARM Cortex-M. Son mécanisme central est la traduction binaire dynamique, qui convertit efficacement le flux d'instructions de la plateforme cible en flux d'instructions de l'hôte. Il est donc fréquemment utilisé pour la transplantation de systèmes d'exploitation et la vérification de logiciels embarqués. Cependant, l'accent de la simulation QEMU est mis sur le noyau CPU et le comportement du système d'exploitation, et sa prise en charge de la modélisation des périphériques intégrés des microcontrôleurs (GPIO, ADC, TIM, etc.) est encore incomplète. De plus, le

processus de configuration et la courbe d'apprentissage ne sont pas adaptés aux scénarios pédagogiques et au développement rapide de prototypes.

Proteus, développé par Labcenter Electronics au Royaume-Uni, est un outil de conception électronique assistée par ordinateur (EDA) intégrant la simulation de circuits, la conception de circuits imprimés et le débogage conjoint des microcontrôleurs. Ce logiciel est compatible avec la simulation de la série STM32 et intègre des instruments virtuels et des oscilloscopes pour l'analyse, ce qui le rend largement utilisé dans les cours d'électronique des universités. Cependant, Proteus utilise un modèle de licence commercial, dont le coût est élevé, et sa vitesse de simulation est contrainte par la précision au niveau du circuit ; lorsque la taille du système augmente, l'efficacité d'exécution diminue de manière significative.

Au niveau national, des universités telles que l'Université Tsinghua et l'Université Zhejiang ont mené des explorations prospectives autour des environnements d'expérimentation virtuels embarqués. Certaines institutions ont construit des plateformes d'expérimentation embarquée à distance basées sur le Web, permettant aux étudiants d'accéder via un navigateur à du matériel réel déployé dans une salle informatique pour réaliser la programmation et le débogage en ligne. Bien que ces solutions permettent le partage des ressources matérielles via le réseau, elles restent fondamentalement liées à des équipements physiques et n'éliminent pas les coûts d'achat matériel ni les contraintes de quantité.

Ces dernières années, la pénétration du langage Python dans la recherche et l'enseignement a continué d'augmenter, et des plateformes pédagogiques embarquées de simulation purement logicielle basées sur des langages de haut niveau ont commencé à attirer l'attention des chercheurs. Ces solutions utilisent Python pour décrire le comportement du noyau MCU et des périphériques, offrant des avantages tels qu'un cycle de développement court, une extension fonctionnelle pratique et une exécution multiplateforme. Cependant, elles présentent encore des écarts en termes de précision temporelle et d'efficacité d'exécution par rapport aux simulateurs de bas niveau.

1.2.2 Recherche sur les systèmes de surveillance environnementale

Au niveau des capteurs et des équipements de surveillance, les fabricants internationaux ont déjà lancé plusieurs générations de produits commercialisés. Les capteurs de température et d'humidité SHT de Sensirion (États-Unis), les capteurs de pression BMP de Bosch (Allemagne) et les capteurs de gaz CCS de AMS (Autriche) représentent les niveaux de technologie avancés de l'industrie. Ces produits se caractérisent généralement par une haute précision de détection, une faible consommation d'énergie et des interfaces numériques standardisées, et sont largement utilisés dans les instruments de surveillance environnementale professionnels.

Les institutions de recherche nationales ont également réalisé des progrès notables dans cette direction. Des unités de recherche telles que l'Académie chinoise des sciences et

l'Université Tsinghua ont mené des études systématiques sur la surveillance des polluants atmosphériques, la détection de la qualité de l'eau et la mesure des paramètres du sol, développant plusieurs dispositifs de surveillance avec des droits de propriété intellectuelle indépendants. Sur le marché de la consommation grand public, les appareils de détection environnementale pour la maison intelligente lancés par des entreprises de terminaux telles que Xiaomi et Huawei intègrent généralement plusieurs types de capteurs et des modules de communication Wi-Fi, permettant aux utilisateurs de consulter les indicateurs environnementaux en temps réel via une application mobile, avec une forte acceptation du marché.

En ce qui concerne les plateformes cloud, des plateformes telles que ThingSpeak et Adafruit IO à l'étranger, ainsi que OneNET et la plateforme IoT d'Alibaba Cloud en Chine, fournissent une infrastructure relativement complète pour le stockage à distance, l'analyse calculatoire et la présentation graphique des données de surveillance. Ces plateformes utilisent généralement le protocole MQTT comme moyen d'accès des terminaux, et intègrent des moteurs de règles et des mécanismes de notification d'événements.

Il convient de noter que les résultats de recherche et les produits commerciaux existants sont principalement orientés vers des scénarios de déploiement réels, accordant une attention insuffisante aux lois cognitives et aux besoins expérimentaux spécifiques au processus d'enseignement. Comment abstraire et virtualiser le processus de développement d'un système de surveillance environnementale pour l'adapter aux objectifs pédagogiques des cours embarqués reste un sujet à explorer en profondeur.

1.3 Contributions principales

Cet article prend la virtualisation de la station intelligente de surveillance environnementale STM32 comme sujet central, et mène la recherche selon les quatre dimensions suivantes :

(1) Conception et réalisation d'une plateforme matérielle virtuelle STM32.

En prenant le STM32F103C8T6 comme prototype, une plateforme matérielle MCU virtuelle complète a été conçue, incluant la simulation du noyau Cortex-M3, le système de cartographie mémoire, le timer SysTick, les GPIO (entrées/sorties générales), l'ADC (convertisseur analogique-numérique), le contrôleur de bus I2C, la communication série UART et les timers généraux TIM. Cette plateforme est implémentée en langage Python, offrant une forte lisibilité du code, une extensibilité facile et une exécution multiplateforme.

(2) Développement de pilotes virtuels de capteurs environnementaux.

Sur la base de la plateforme matérielle virtuelle, des pilotes virtuels pour cinq capteurs environnementaux courants ont été développés : DHT11, MQ-135, BH1750, BMP280 et le capteur de bruit. Un cadre de modélisation unifié des capteurs a été établi, incluant la définition des paramètres environnementaux, le modèle de conversion des grandeurs

physiques et le modèle de bruit gaussien, de sorte que les données simulées respectent les lois physiques tout en présentant un certain caractère aléatoire, se rapprochant ainsi davantage des caractéristiques de sortie des capteurs réels.

(3) **Construction d'un système de visualisation Web des données.** Le backend a été développé avec le framework Python Flask, utilisant la base de données SQLite pour stocker les données des capteurs, les enregistrements d'alarmes et les journaux système. Une interface API RESTful a été conçue pour être consommée par le frontend. Le frontend utilise des technologies natives HTML/CSS/JavaScript, combinées à la bibliothèque de graphiques Chart.js, pour réaliser l'affichage de graphiques linéaires en temps réel des données des capteurs, le résumé des informations statistiques et le panneau d'alarmes.

(4) **Mécanisme de contrôle et d'interaction des paramètres environnementaux.** Un mécanisme de communication interprocessus (IPC) basé sur le partage de fichiers a été conçu, réalisant une communication bidirectionnelle entre le panneau de contrôle Web et le MCU virtuel via le fichier `env_control.txt`. Les utilisateurs peuvent ajuster les paramètres environnementaux (température, humidité, pression, etc.) en temps réel via l'interface Web, et les capteurs virtuels génèrent les données simulées correspondantes en fonction des paramètres mis à jour, réalisant ainsi une interaction en boucle fermée.

1.4 Structure du document

Le document est organisé en sept chapitres, dont le contenu est structuré comme suit :

Le chapitre 1 est l'introduction, présentant le contexte et l'importance de la recherche, l'état de l'art, les contributions principales et la structure du document.

Le chapitre 2 présente la conception générale du système, analysant les besoins fonctionnels et non fonctionnels, concevant l'architecture globale en quatre couches, et introduisant les choix technologiques clés.

Le chapitre 3 traite de la conception de la plateforme matérielle virtuelle STM32, détaillant les principes de simulation et les méthodes de réalisation du cœur MCU virtuel, du système mémoire et des divers périphériques.

Le chapitre 4 concerne les pilotes de capteurs et l'acquisition de données, introduisant le cadre de modélisation général pour la virtualisation des capteurs, et analysant en détail les principes de simulation et la réalisation des pilotes de chaque capteur.

Le chapitre 5 présente la conception et la réalisation du logiciel, incluant les modules de stockage de données, de système d'alarmes, de visualisation Web et de panneau de contrôle.

Le chapitre 6 porte sur les tests et la validation du système, effectuant des tests fonctionnels, des tests de performance et la vérification de la précision des capteurs, avec une analyse des résultats de test.

Le chapitre 7 est la conclusion et les perspectives, récapitulant les travaux principaux de cet article et proposant des directions d'amélioration futures.

2 Conception générale du système

2.1 Analyse des besoins fonctionnels

2.1.1 Besoins fonctionnels

Ce système doit réaliser une plateforme de virtualisation d'une station intelligente de surveillance environnementale, dont les objectifs fonctionnels couvrent les six aspects suivants :

(1) **Simulation du MCU virtuel** : le système doit être capable de simuler les fonctions principales du microcontrôleur STM32F103C8T6, incluant l'exécution des instructions du processeur, l'accès mémoire, le système d'horloge et le contrôle des périphériques. Le code embarqué en C (ou le code Python équivalent) écrit par l'utilisateur doit pouvoir s'exécuter correctement sur la plateforme virtuelle sans modification pour être porté vers le matériel réel.

(2) **Acquisition de données multi-capteurs** : le système doit supporter l'acquisition de données d'au moins cinq capteurs environnementaux, incluant la température, l'humidité, la qualité de l'air (exprimée par la concentration de gaz nocifs), l'intensité lumineuse, la pression atmosphérique et le niveau sonore. Chaque capteur doit disposer d'un pilote indépendant, respectant le protocole de communication correspondant.

(3) **Stockage et gestion des données** : le système doit être capable de stocker de manière persistante les données des capteurs acquises dans une base de données, supporter la consultation par plage de temps, l'analyse statistique et l'exportation de données. En parallèle, il doit enregistrer les journaux d'exécution du système et les événements d'alarmes, facilitant le dépannage et l'audit.

(4) **Visualisation des données en temps réel** : le système doit fournir une interface Web d'affichage des données en temps réel, présentant intuitivement les lectures actuelles et les tendances historiques de chaque capteur sous forme de cartes de valeurs et de graphiques linéaires. Les graphiques doivent supporter le zoom sur l'axe temporel et les info-bulles sur les points de données.

(5) **Fonction d'alarme intelligente** : le système doit supporter la configuration de seuils supérieurs et inférieurs pour chaque capteur. Lorsqu'une lecture de capteur dépasse la plage de seuils, une alarme est automatiquement déclenchée et l'information d'alarme est enregistrée dans la base de données et le panneau frontend. L'état d'alarme doit pouvoir être mis à jour et effacé en temps réel.

(6) **Contrôle des paramètres environnementaux** : le système doit permettre à l'utilisateur d'ajuster les paramètres environnementaux en temps réel via l'interface Web,

simulant le comportement des capteurs dans différentes conditions environnementales. Les commandes de contrôle doivent prendre effet rapidement et les valeurs actuelles des paramètres environnementaux doivent être affichées en retour sur l'interface.

2.1.2 Besoins non fonctionnels

En dehors des objectifs fonctionnels, le fonctionnement du système doit également respecter les contraintes suivantes :

(1) **Besoins de performance** : l'efficacité d'exécution des instructions du MCU virtuel doit être suffisamment élevée, le temps de traitement d'un cycle d'acquisition de données ne dépassant pas 1 seconde. Le temps de réponse des API Web doit être contrôlé en dessous de 100 millisecondes, et la fréquence de rafraîchissement des graphiques frontend ne doit pas être inférieure à 1 Hz.

(2) **Besoins de fiabilité** : le système doit pouvoir fonctionner de manière continue et stable pendant au moins 72 heures sans plantage. Les opérations de base de données doivent disposer d'un mécanisme de protection transactionnelle pour éviter la perte ou l'incohérence des données. Les pilotes de capteurs doivent être capables de gérer des situations anormales telles que les délais de communication et les erreurs de vérification.

(3) **Besoins d'évolutivité** : l'architecture du système doit présenter une conception modulaire de bonne qualité. L'ajout de nouveaux types de capteurs ou de modules périphériques doit pouvoir se faire en ajoutant des fichiers de pilotes et des fichiers de configuration, sans modifier le code cœur. La conception des API du backend Web doit suivre les spécifications RESTful, facilitant l'extension des fonctionnalités frontend.

(4) **Besoins de facilité d'utilisation** : le système doit fournir une documentation et des instructions d'utilisation claires. L'interface Web doit avoir une disposition raisonnable et des opérations intuitives. Les interfaces API du matériel virtuel doivent être conçues de manière concise, réduisant les coûts d'apprentissage et de développement des utilisateurs.

(5) **Besoins de portabilité** : le système doit pouvoir fonctionner sur les systèmes d'exploitation principaux tels que Windows, Linux et macOS, sans dépendre de caractéristiques matérielles ou système spécifiques. La plateforme virtuelle elle-même doit être implémentée en langage Python, utilisant la portabilité multiplateforme de Python pour garantir la compatibilité.

2.2 Architecture globale

Sur la base des besoins précédents, ce système est divisé en quatre couches de bas en haut : couche d'abstraction matérielle, couche de pilotes de périphériques, couche de logique métier et couche de présentation applicative. Les frontières fonctionnelles de chaque couche sont présentées dans le Tableau 1.

Table 1: Conception de l'architecture en quatre couches du système

Couche	Nom	Description des responsabilités
Couche 4	Couche de présentation applicative	Responsable du rendu de l'interface frontend Web, du tracé de graphiques, de l'interaction utilisateur et de l'affichage des données, implémenté en HTML/CSS/JavaScript natif
Couche 3	Couche de logique métier	Responsable du service backend Flask, de la conception des API RESTful, de la gestion du stockage des données, du traitement de la logique d'alarmes et de l'analyse des commandes de contrôle
Couche 2	Couche de pilotes de périphériques	Responsable du développement des pilotes de divers capteurs et de la réalisation de la boucle d'acquisition de données, convertissant les grandeurs physiques en grandeurs numériques
Couche 1	Couche d'abstraction matérielle	Responsable de la simulation du cœur MCU virtuel, du système mémoire, des contrôleurs de bus et des divers périphériques, fournissant une interface au niveau des registres matériels

La couche d'abstraction matérielle (Hardware Abstraction Layer, HAL) constitue la fondation de l'ensemble de la plateforme, et son comportement au niveau des registres est cohérent avec celui de la puce STM32 réelle. La couche de pilotes de périphériques appelle les interfaces exposées par la HAL pour accomplir l'initialisation des capteurs, la lecture des données et l'analyse des protocoles. La couche de logique métier assume les tâches de traitement des données brutes, de stockage persistant et d'analyse, et ouvre ses capacités à la couche supérieure sous forme d'API standardisées. La couche de présentation applicative se concentre sur l'optimisation de la qualité de présentation de l'interface et de la fluidité de l'interaction.

Les couches collaborent entre elles via des contrats d'interface normalisés ; les modules de couche supérieure sont interdits d'opérer directement sur l'état interne des couches inférieures en contournant les interfaces. Cette conception contrôle efficacement le degré de couplage entre les modules. Lorsqu'il est nécessaire d'introduire un nouveau capteur, il suffit de compléter le fichier de pilote correspondant dans la couche de pilotes de périphériques et d'enregistrer la règle d'analyse dans la couche de logique métier ; ni la

couche d'abstraction matérielle ni la couche de présentation applicative ne nécessitent de modification.

2.3 Choix technologiques

Dans la sélection des outils et des frameworks, ce système suit le principe de “simplicité, efficacité et facilité d'extension”. Les combinaisons de technologies clés sont résumées dans le Tableau 2.

Table 2: Choix technologiques du système

Catégorie	Technologie	Justification du choix
Langage de développement	Python 3.11	Syntaxe concise, haute efficacité de développement, riches bibliothèques standard et tierces, bon support multiplateforme
Framework Web	Flask 3.0	Léger, flexible, facile à prendre en main, communauté active, extensions riches
Base de données	SQLite 3	Zéro configuration, stockage en fichier unique, pas besoin de processus serveur indépendant, adapté aux applications de petite et moyenne taille
Technologies frontend	HTML/CSS/JS natif	Aucune dépendance de build, chargement rapide, bonne compatibilité, facile à déployer
Bibliothèque de graphiques	Chart.js 4.x	Open source et gratuit, API conviviale, supporte le design responsive et les animations
Gestion de versions	Git	Gestion de versions distribuée, gestion de branches flexible, haute efficacité de collaboration

Le choix de Python 3.11 comme langage de développement de base repose sur trois considérations : premièrement, la syntaxe de Python est concise et la lisibilité du code source est remarquable, facilitant l'explication ligne par ligne dans les scénarios pédagogiques ; deuxièmement, la bibliothèque standard couvre les tâches courantes telles que la communication réseau, l'accès aux bases de données, la manipulation de fichiers et la programmation concurrente, réduisant l'introduction de dépendances externes ; troisièmement, le mode d'exécution interprété élimine le temps d'attente de compilation, permettant une validation immédiate après modification et réduisant le cycle de débogage.

Flask est un framework Web léger basé sur Python, dont la philosophie met l'accent sur la simplicité et la flexibilité. Par rapport aux frameworks full-stack tels que Django, Flask n'impose pas de structure de répertoire de projet ni de relations de liaison entre composants, permettant aux développeurs de choisir la base de données, le moteur de templates et la stratégie d'authentification selon les besoins réels. Pour une application Web de taille modérée comme ce système, la couverture fonctionnelle de Flask se situe dans une plage raisonnable, évitant à la fois la surconception et l'absence de capacités.

SQLite est une base de données relationnelle embarquée où l'ensemble des données est concentré dans un unique fichier disque, sans nécessité de processus serveur indépendant. Cette propriété "prêt à l'emploi" simplifie considérablement le processus de déploiement, en particulier pour le développement individuel, les expériences pédagogiques et les petites applications. SQLite est compatible avec la syntaxe SQL standard et les transactions ACID, ce qui est suffisant pour répondre aux exigences de persistance et d'interrogation analytique de ce système.

La stratégie de technologies frontend a choisi le HTML5, le CSS3 et le JavaScript natifs, sans introduire de frameworks componentisés tels que React ou Vue. La motivation principale de cette décision est de contrôler la complexité globale, d'éviter les surcoûts de configuration de la chaîne d'outils de build, et de maintenir le code frontend dans une forme directement lisible. Chart.js, en tant que bibliothèque de composants graphiques open source, offre une diversité de types de graphiques et de dimensions de configuration. Son moteur de rendu basé sur Canvas offre de bonnes performances, répondant aux exigences de performance de la visualisation des données en temps réel.

3 Conception de la plateforme matérielle virtuelle STM32

La plateforme matérielle virtuelle STM32 constitue le pilier sous-jacent de l'ensemble du système. L'idée de sa construction est de reproduire dans un environnement purement logiciel le comportement au niveau des registres du microcontrôleur STM32F103C8T6, permettant aux pilotes de capteurs et au code de la couche application de s'exécuter comme sur une puce réelle. Les sections suivantes détaillent respectivement la simulation du noyau MCU, le mécanisme de cartographie mémoire et la simulation des principaux périphériques.

3.1 Conception du cœur MCU virtuel

3.1.1 Vue d'ensemble des paramètres du STM32F103C8T6

Le STM32F103C8T6, produit par STMicroelectronics, est équipé d'un cœur ARM Cortex-M3 et appartient au modèle de densité moyenne de la famille STM32F1. Cette puce offre un bon équilibre entre performance et coût, et est donc largement utilisée dans l'enseignement universitaire et la pratique industrielle. Le Tableau 3 résume ses principaux indicateurs techniques.

Table 3: Principaux paramètres techniques du STM32F103C8T6

Paramètre	Spécification
Architecture du cœur	ARM Cortex-M3
Fréquence maximale	72 MHz
Mémoire Flash	64 Ko
SRAM	20 Ko
Broches GPIO	37 (boîtier LQFP48)
ADC	2 convertisseurs 12 bits, 16 canaux
Timers	3 timers généraux + 1 timer avancé + 2 timers de base
Interfaces de communication	2 I2C, 3 USART, 2 SPI, 1 CAN
Tension de fonctionnement	2,0 V – 3,6 V
Température de fonctionnement	−40°C – +85°C
Type de boîtier	LQFP48

Étant donné que cette plateforme est implémentée en Python, elle ne simule pas le jeu d'instructions ARM ni les cycles d'horloge au cycle près, mais se concentre sur la reproduction fidèle du comportement des registres périphériques. La base de temps utilise le milliseconde comme plus petite unité, et les délais et la minuterie sont réalisés à l'aide du module `time` de la bibliothèque standard Python.

3.1.2 Simulation du timer SysTick

Le cœur Cortex-M3 intègre SysTick, un compteur décroissant 24 bits fournissant une cadence périodique au système d'exploitation et aux applications. Dans la plateforme virtuelle implémentée en Python, la simulation de SysTick se concentre sur trois capacités : le décrémentation du compteur, le réglage de la valeur de rechargement et le déclenchement d'interruptions. En raison des caractéristiques d'exécution du langage interprété, la précision temporelle au niveau de la microseconde est difficile à garantir ; une solution approximative basée sur le temps relatif est donc adoptée.

La fonction `delay_ms` assume la responsabilité du délai au niveau de la milliseconde. Cette fonction enregistre l'heure système actuelle à l'entrée, puis interroge en boucle le temps écoulé ; une fois que le nombre de millisecondes défini est atteint, elle retourne. Cette stratégie d'attente active occupe le CPU, mais elle est suffisante pour les démonstrations pédagogiques et la vérification fonctionnelle.

```
1 import time
2
3 def delay_ms(ms):
4     """
5     Fonction de délai en millisecondes
6     Paramètres :
7         ms : nombre de millisecondes de délai
8     """
9     start = time.time()
10    while (time.time() - start) * 1000 < ms:
11        pass
```

Listing 1: Implémentation de la fonction de délai virtuelle `delay_ms`

Le Code 1 présente l'implémentation par attente active de `delay_ms`. Au cours du processus pédagogique, les étudiants peuvent être invités à discuter des avantages et inconvénients de cette solution : sa simplicité d'implémentation et son absence de dépendances externes sont ses forces ; l'incapacité du CPU à traiter d'autres tâches pendant la période de délai est son point faible évident. Pour l'améliorer, on peut essayer de réécrire une version non bloquante à l'aide de `threading` ou `asyncio`.

3.2 Conception du système mémoire

3.2.1 Modèle de cartographie mémoire

Le STM32F103C8T6 mappe uniformément la Flash, la SRAM et les registres des périphériques dans un espace d'adressage 32 bits. Le système mémoire virtuel reproduit par conséquent trois zones d'adresses clés :

(1) **Mémoire Flash** : utilisée pour stocker le code du programme et les données constantes, la plage d'adresses commence à `0x08000000`, avec une taille de 64 Ko. Dans la plateforme virtuelle, la zone Flash sert principalement à stocker le code du firmware simulé et les paramètres de configuration.

(2) **SRAM** : utilisée pour stocker les variables d'exécution, la pile et le tas, la plage d'adresses commence à `0x20000000`, avec une taille de 20 Ko. La zone SRAM de la plateforme virtuelle est implémentée sous forme de tableau d'octets, supportant les accès en lecture/écriture par octet, demi-mot et mot.

(3) **Registres des périphériques** : incluant les registres de configuration et de données des périphériques tels que GPIO, ADC, UART, TIM, I2C, la plage d'adresses commence à 0x40000000. Chaque périphérique occupe un espace d'adresses continu, et les registres sont disposés selon des offsets fixes.

La classe `MemoryRegion` est le cœur du système mémoire virtuel, gérant le flux d'octets de lecture/écriture d'un espace d'adresses continu. Le STM32 adopte le mode little-endian : l'octet de poids faible d'une donnée multi-octet est stocké à l'adresse basse, et l'octet de poids fort à l'adresse haute. La classe `MemoryRegion` respecte strictement cette règle d'ordonnancement des octets lors des accès.

3.2.2 Implémentation du mode little-endian

Le mode little-endian signifie que l'octet de poids fort d'une donnée est stocké à l'adresse haute, et l'octet de poids faible à l'adresse basse. Prenant 0x12345678 comme exemple, les octets en mémoire sont successivement 0x78, 0x56, 0x34, 0x12, avec les adresses croissantes du bas vers le haut.

Le module `struct` de Python fournit des moyens de contrôle de l'ordre des octets : le caractère de format '`<`' indique le little-endian, et '`>`' le big-endian. La classe `MemoryRegion` utilise `struct.pack` et `struct.unpack` pour accomplir la sérialisation et la désérialisation des données multi-octets, garantissant ainsi que la plateforme virtuelle et le STM32 réel se comportent de manière cohérente en termes de gestion de l'ordre des octets.

3.2.3 Cartographie des registres

`RegisterMap` ajoute une couche d'encapsulation au-dessus de `MemoryRegion`, orientée vers la gestion des registres de périphériques. Chaque instance de périphérique contient un objet `RegisterMap`, permettant au développeur d'accéder aux registres par leur nom plutôt que par leur adresse brute, améliorant ainsi la lisibilité du code.

`RegisterMap` fournit principalement quatre capacités :

(1) **Enregistrement des registres** : lors de l'initialisation, définir l'ensemble des registres appartenant à ce périphérique, incluant le nom du registre, l'adresse de décalage, la valeur par défaut et les droits d'accès (lecture seule / écriture seule / lecture-écriture).

(2) **Accès aux champs de bits** : supporter les opérations de lecture et d'écriture sur un bit ou un champ de bits individuel dans un registre. Par exemple, on peut mettre le bit 0 du registre CR1 à 1 via `set_bit('CR1', 0)`, sans avoir besoin de lire l'intégralité du registre puis de calculer manuellement.

(3) **Protection en écriture** : pour les registres en lecture seule (comme le registre d'état SR), `RegisterMap` refuse les opérations d'écriture directes, empêchant le code de couche supérieure de modifier accidentellement des valeurs de registres qui ne devraient pas être modifiées.

(4) **Rappel de changement** : permettre d'enregistrer des fonctions de rappel pour des registres spécifiques, déclenchées automatiquement lorsque la valeur du registre change. Ce mécanisme est très important pour simuler la réaction comportementale des périphériques ; par exemple, lorsque le registre ODR du GPIO est écrit, l'état de sortie de la broche doit être mis à jour immédiatement.

3.3 Simulation des périphériques

3.3.1 GPIO (entrées/sorties générales)

Le GPIO est responsable de l'entrée et de la sortie des signaux numériques, et constitue la première interface d'interaction entre le microcontrôleur et les circuits externes. Les ports GPIO du STM32F103C8T6 comportent 16 broches par groupe (Pin 0 à Pin 15), configurables individuellement en mode entrée flottante, entrée pull-up, entrée pull-down, entrée analogique ou sortie push-pull / drain ouvert.

Le GPIO virtuel s'articule autour des mécanismes suivants :

(1) **Tableau d'états des broches** : chaque port GPIO maintient un tableau de longueur 16, enregistrant l'état de niveau logique actuel de chaque broche (0 ou 1). En mode entrée, l'état de la broche est déterminé par le périphérique externe ou les résistances de pull-up/pull-down ; en mode sortie, l'état de la broche est déterminé par la valeur du registre ODR.

(2) **Registre de données d'entrée IDR** : IDR (Input Data Register) sert à lire les niveaux d'entrée actuels des broches du port GPIO. Dans la plateforme virtuelle, la valeur de l'IDR est obtenue par encodage du tableau d'états des broches. Plus précisément, le i -ème bit de l'IDR est égal à l'état de niveau logique actuel de la broche i .

La formule de calcul de l'IDR est :

$$\text{IDR} = \sum_{i=0}^{15} P_i \times 2^i \quad (1)$$

où P_i représente l'état de niveau logique de la broche i , avec $P_i \in \{0, 1\}$.

(3) **Registre de données de sortie ODR** : ODR (Output Data Register) sert à définir les niveaux de sortie des broches du port GPIO. Lorsqu'une nouvelle valeur est écrite dans l'ODR, le module GPIO virtuel analyse bit par bit, met à jour l'état de sortie de la broche correspondante, et déclenche les fonctions de rappel enregistrées.

(4) **Mécanisme de rappel** : pour supporter la réaction des périphériques externes aux changements d'état du GPIO, le GPIO virtuel implémente un mécanisme d'enregistrement de rappels. L'utilisateur peut enregistrer des fonctions de rappel pour des événements de front montant, de front descendant ou de changement de niveau sur une broche spécifique. Lorsque l'état de la broche change de manière correspondante, toutes les fonctions de rappel enregistrées sont exécutées séquentiellement dans l'ordre de leur enregistrement.

Ce mécanisme est très utile pour simuler la sortie d'interruption des capteurs et le contrôle de commutation des indicateurs LED.

3.3.2 ADC (convertisseur analogique-numérique)

L'ADC mappe une tension analogique continue en une quantité numérique discrète, jouant un rôle de traduction entre le front-end analogique et le back-end numérique. Le STM32F103C8T6 intègre deux convertisseurs SAR 12 bits, supportant 16 entrées externes ainsi que 2 canaux internes (capteur de température interne et tension de référence).

Les attributs clés de l'ADC virtuel sont les suivants :

(1) **Résolution 12 bits** : le résultat de conversion de l'ADC est un entier non signé 12 bits, avec une plage de valeurs de 0 à 4095. La formule de quantification correspondante est :

$$D = \left\lfloor \frac{V_{in}}{V_{ref}} \times 4096 \right\rfloor \quad (2)$$

où D est la sortie numérique, V_{in} est la tension analogique d'entrée, et V_{ref} est la tension de référence (généralement 3,3 V).

(2) **Conversion tension-numérique** : lorsque le code de couche supérieur lit un canal via l'ADC, l'ADC virtuel calcule la quantité numérique correspondante selon la formule 2 en fonction de la valeur de tension analogique actuellement connectée à ce canal. La valeur de tension analogique est fournie par le capteur virtuel connecté ou par la source de signal externe.

(3) **Modèle de bruit** : pour simuler plus fidèlement les caractéristiques de conversion de l'ADC, un bruit blanc gaussien est superposé au résultat de conversion idéal. L'amplitude du bruit peut être ajustée via des paramètres de configuration, avec une valeur par défaut de ± 2 LSB (Least Significant Bit). La formule de conversion avec bruit est :

$$D_{actual} = D_{ideal} + N(0, \sigma^2) \quad (3)$$

où $N(0, \sigma^2)$ représente un nombre aléatoire distribué normalement avec une moyenne de 0 et une variance de σ^2 , la valeur par défaut de σ étant d'environ 2.

(4) **Sélection et activation des canaux** : l'ADC virtuel supporte la configuration multi-canaux, définissant l'ordre et la quantité des canaux de conversion via le registre de séquence SQR (Regular Sequence Register). Avant de lancer une conversion, il est nécessaire d'activer l'ADC (mettre le bit ADON du registre CR2 à 1), puis de démarrer la conversion (mettre le bit SWSTART du registre CR2 à 1).

3.3.3 Contrôleur de bus I2C

Le bus I2C accomplit la communication multi-maître et multi-esclave via deux lignes SDA et SCL, et est très largement utilisé dans les capteurs, les EEPROM et autres dispositifs.

Le STM32F103C8T6 est équipé de deux interfaces I2C, compatibles avec le mode standard (100 kHz) et le mode rapide (400 kHz).

L'I2C virtuelle se concentre sur la simulation des comportements suivants :

(1) **Attachement des périphériques** : le bus I2C virtuel maintient une liste des périphériques attachés, chaque périphérique étant identifié de manière unique par une adresse d'esclave 7 bits. Lorsque le code de couche supérieur envoie un signal de démarrage et un octet d'adresse via l'I2C, le contrôleur de bus recherche l'instance du périphérique virtuel correspondante en fonction de l'adresse, et établit la connexion de communication.

L'interface d'attachement des périphériques est définie comme suit :

```
1 def attach_device(self, address, device):
2     """
3     Attache un périphérique virtuel au bus I2C
4     Paramètres :
5         address : adresse d'esclave sur 7 bits
6         device  : instance du périphérique virtuel, doit implémenter
7                   les méthodes read/write
8     """
9     self.devices[address] = device
```

Listing 2: Interface d'attachement des périphériques I2C virtuels

(2) **Opérations de lecture et d'écriture** : l'I2C virtuelle supporte les opérations de lecture et d'écriture au niveau de l'octet et du tampon. Lors d'une opération d'écriture, le maître envoie l'adresse de l'esclave et le bit d'indication d'écriture (0), puis envoie l'adresse du registre à écrire et les octets de données ; lors d'une opération de lecture, le maître envoie d'abord l'adresse de l'esclave et le bit d'indication d'écriture pour spécifier l'adresse du registre, puis renvoie un signal de démarrage et l'adresse de l'esclave avec le bit d'indication de lecture (1), et l'esclave retourne l'octet de données demandé.

(3) **Gestion des erreurs** : l'I2C virtuelle implémente un mécanisme de détection d'erreurs de base, incluant : absence d'acquittement d'adresse (NACK) – lorsque l'adresse de l'esclave envoyée ne trouve pas de correspondance dans la liste des périphériques, une erreur est retournée ; bus occupé – lorsque plusieurs maîtres tentent simultanément de contrôler le bus, une détection de conflit est générée ; vérification des données – supporte la vérification CRC des données transmises (optionnelle).

3.3.4 Communication série UART

L'UART, en tant qu'interface série asynchrone, est souvent utilisée pour l'échange de données entre le MCU et un PC, un module Bluetooth, un récepteur GPS, etc. Le STM32F103C8T6 dispose de trois USART indépendants, supportant le full-duplex et

plusieurs réglages de débit en bauds.

L'implémentation de l'UART virtuelle se concentre sur les points suivants :

(1) **Configuration du débit en bauds** : le débit en bauds représente le nombre de bits transmis par seconde, et constitue un paramètre clé de la communication UART. L'UART virtuelle enregistre la valeur du débit en bauds actuellement configurée. Bien qu'en raison de la nature de la simulation logicielle il soit impossible de contrôler véritablement la synchronisation au niveau du bit selon le débit en bauds, cette valeur influence certains comportements de simulation liés au timing (tels que les réglages de délai d'attente). Les valeurs de débit en bauds courantes incluent 9600, 115200, 921600, etc.

(2) **Simulation de la redirection de printf** : dans le développement embarqué, la fonction de sortie standard `printf` est généralement redirigée vers l'UART afin de sortir les informations de débogage via le port série. L'UART virtuelle fournit une interface d'envoi de chaînes équivalente, mettant en cache la chaîne à envoyer dans un tampon interne et déclenchant la fonction de rappel du récepteur.

(3) **Mécanisme de rappel** : l'UART virtuelle supporte l'enregistrement d'une fonction de rappel de réception, appelée automatiquement lorsque des données arrivent. Ce mécanisme simule l'interruption de réception de l'UART réel, permettant au code de couche supérieure de traiter les données du port série de manière événementielle, sans avoir besoin d'interroger le tampon de réception.

```
1 class VirtualUART:
2     def __init__(self):
3         self.baudrate = 115200
4         self.rx_callback = None
5         self.tx_buffer = []
6
7     def set_rx_callback(self, callback):
8         """Enregistre la fonction de rappel de réception"""
9         self.rx_callback = callback
10
11     def send(self, data):
12         """Envoie des données (chaîne ou octets)"""
13         if isinstance(data, str):
14             data = data.encode('utf-8')
15         self.tx_buffer.extend(data)
16         if self.rx_callback:
17             self.rx_callback(data)
18
19     def printf(self, fmt, *args):
20         """Sortie formatée, simulant printf"""
21         msg = fmt % args
```

22

```
self.send(msg)
```

Listing 3: Enregistrement de rappel et envoi UART virtuels

Le Code 3 présente l'implémentation de la classe `VirtualUART`. L'interface `send` est compatible avec les types `str` et `bytes`, tandis que `printf` reproduit la sémantique de la sortie formatée du langage C.

3.3.5 TIM (timer général)

Le TIM est le périphérique le plus polyvalent de la famille STM32, pouvant servir de timer, de compteur, de générateur de signal PWM ou de capture d'entrée externe. Ce projet utilise uniquement sa capacité de timing de base, fournissant une source de déclenchement à fréquence fixe pour l'échantillonnage des capteurs.

Le `VirtualTIM` utilise le thread démon du module `threading` de Python pour accomplir le chronométrage en arrière-plan. Le thread démon se termine automatiquement lorsque le thread principal se termine, sans nécessiter de nettoyage manuel, ce qui correspond parfaitement aux besoins des tâches périodiques en arrière-plan.

La logique de fonctionnement du TIM virtuel peut être divisée en trois étapes :

(1) **Incrémenter le compteur** : après le démarrage du TIM, le thread démon incrémente le compteur interne à intervalles de temps fixes (déterminés par le facteur de prédiviseur et la valeur de rechargement automatique). La fréquence d'incrémenter du compteur est calculée selon la formule :

$$f_{count} = \frac{f_{clk}}{(PSC + 1) \times (ARR + 1)} \quad (4)$$

où f_{clk} est la fréquence d'horloge du timer (généralement 72 MHz), PSC est la valeur du prédiviseur, et ARR est la valeur de rechargement automatique.

(2) **Rappel périodique** : lorsque le compteur atteint la valeur de rechargement automatique, il est remis à zéro et un événement de mise à jour (Update Event) est déclenché. Si une fonction de rappel périodique est enregistrée, elle est appelée à ce moment. Dans le système de surveillance environnementale, la fonction de rappel périodique exécute généralement la tâche d'acquisition de données des capteurs, réalisant ainsi un échantillonnage automatique à fréquence fixe.

(3) **Démarrage et arrêt** : le TIM est contrôlé par le bit CEN du registre CR1 pour le démarrage et l'arrêt. Mettre le bit CEN à 1 démarre le timer, et le remettre à zéro l'arrête. Le TIM virtuel crée et démarre le thread démon lors du démarrage, et définit un indicateur d'événement pour notifier le thread de se terminer lors de l'arrêt.

```
1 import threading
2 import time
```



```
3
4 class VirtualTIM:
5     def __init__(self):
6         self.prescaler = 7199          # PSC
7         self.auto_reload = 9999        # ARR
8         self.counter = 0
9         self.running = False
10        self.period_callback = None
11        self._stop_event = threading.Event()
12
13    def set_period_callback(self, callback):
14        """Définit la fonction de rappel périodique"""
15        self.period_callback = callback
16
17    def _timer_thread(self):
18        """Thread démon du timer"""
19        period_ms = (self.prescaler + 1) * (self.auto_reload + 1) /
20                    72000
21        while not self._stop_event.is_set():
22            time.sleep(period_ms / 1000.0)
23            self.counter = 0
24            if self.period_callback:
25                self.period_callback()
26
27    def start(self):
28        """Démarrage le timer"""
29        self.running = True
30        self._stop_event.clear()
31        t = threading.Thread(target=self._timer_thread, daemon=True)
32        t.start()
33
34    def stop(self):
35        """Arrête le timer"""
36        self.running = False
37        self._stop_event.set()
```

Listing 4: Implémentation du thread démon du timer virtuel TIM

Le Code 4 montre comment le VirtualTIM utilise le thread démon pour réaliser le déclenchement périodique. Le `period_callback` est exécuté à chaque débordement de période. Modifier `prescaler` et `auto_reload` permet de redéfinir la période de tempori-

sation, correspondant ainsi à divers taux d'échantillonnage.

4 Pilotes de capteurs et acquisition de données

Les capteurs sont l'équivalent des organes sensoriels d'un système de surveillance, transformant les divers paramètres de l'environnement physique en signaux électriques pour traitement par le MCU. Dans un environnement de simulation purement logicielle, le modèle de capteur sert de couche d'interface entre le matériel virtuel et l'application de couche supérieure, et son degré de réalisme influence directement la crédibilité de l'ensemble du système. Nous discutons d'abord de l'idée de modélisation générale, puis analysons un par un les principes de simulation et le codage des pilotes des capteurs spécifiques.

4.1 Principes de virtualisation des capteurs

4.1.1 Cadre de modélisation général

Ce système a établi un cadre de modélisation unifié pour la simulation des capteurs, réduisant tous les types de capteurs à trois maillons : entrée des paramètres environnementaux, modèle de conversion des grandeurs physiques, et sortie numérique. La Fig. ?? donne une illustration conceptuelle de ce cadre.

Ce cadre comprend trois éléments clés :

(1) **Définition des paramètres environnementaux** : chaque capteur est associé à un ensemble de variables de paramètres environnementaux représentant les valeurs réelles des grandeurs physiques mesurées par le capteur. Par exemple, le capteur de température est associé au paramètre `temperature`, et le capteur de luminosité au paramètre `illuminance`. Les valeurs des paramètres environnementaux peuvent être ajustées en temps réel via le panneau de contrôle, simulant différentes conditions environnementales.

(2) **Modèle de conversion des grandeurs physiques** : décrit comment les paramètres environnementaux sont convertis en grandeurs physiques de sortie du capteur (généralement une tension, une résistance ou une quantité numérique). Les modèles de conversion varient selon les capteurs, et peuvent être des relations linéaires, des relations en loi de puissance ou des fonctions par morceaux. Le modèle de conversion doit être établi sur la base de la fiche technique (Datasheet) du capteur et des principes physiques, afin d'assurer la rationalité des données simulées.

(3) **Modèle de bruit** : la sortie des capteurs réels contient inévitablement un bruit aléatoire, dont les sources incluent le bruit thermique, le bruit de grenaille et les interférences électromagnétiques. Pour renforcer le réalisme de la simulation, un bruit blanc gaussien est superposé à la sortie idéale du capteur. L'expression mathématique du modèle de bruit est :

$$Y = X + N(0, \sigma^2) \quad (5)$$

où Y est la valeur de sortie réelle du capteur, X est la valeur de sortie idéale (calculée par le modèle de conversion des grandeurs physiques), et $N(0, \sigma^2)$ est un bruit aléatoire gaussien de moyenne 0 et de variance σ^2 .

L'intensité du bruit σ est déterminée en fonction de la classe de précision du capteur et du scénario d'application typique. Pour les capteurs de haute précision (comme le BMP280), σ prend une valeur plus faible ; pour les capteurs de faible précision (comme le MQ-135), σ prend une valeur plus élevée. Dans des scénarios spéciaux, il est également possible d'ajouter des valeurs aberrantes (Outlier) pour simuler des pannes de capteur ou des changements brutaux de l'environnement.

4.1.2 Interface des pilotes de capteurs

Les pilotes de capteurs virtuels partagent tous le même contrat d'interface ; le code de couche supérieure peut les appeler sans se soucier des différences sous-jacentes. La définition de l'interface est la suivante :

- `init()` : initialise le capteur, configure les paramètres de communication et le mode de fonctionnement.
- `read()` : lit la valeur de mesure actuelle du capteur, retourne les données brutes ou la grandeur physique calibrée.
- `calibrate()` : effectue l'opération de calibration, met à jour les coefficients de calibration (le cas échéant).
- `set_environment(env_dict)` : met à jour les paramètres environnementaux associés au capteur.

Grâce à la cohérence de l'interface, l'introduction d'un nouveau capteur ne nécessite que l'écriture de la classe de pilote correspondante et la complétion des quatre méthodes, tandis que la boucle d'acquisition de données et la couche de présentation Web restent inchangées.

4.2 Capteur de température et d'humidité DHT11

4.2.1 Vue d'ensemble du capteur

Le DHT11 intègre la détection de la température et de l'humidité dans une seule puce, communiquant avec l'hôte via un protocole one-wire. Sa caractéristique la plus marquante est son faible coût, sa petite taille et sa facilité de prise en main, ce qui lui confère une très forte présence dans les projets IoT d'initiation. Le Tableau 4 liste ses principales spécifications.

Table 4: Principaux paramètres techniques du DHT11

Paramètre	Spécification
Plage de mesure (humidité)	20% – 90% RH
Précision de mesure (humidité)	$\pm 5\%$ RH
Plage de mesure (température)	0°C – 50°C
Précision de mesure (température)	$\pm 2^\circ\text{C}$
Résolution	1% RH / 1°C
Période d'échantillonnage	≥ 1 s
Tension d'alimentation	3,3 V – 5,5 V
Protocole de communication	One-wire série

4.2.2 Simulation du protocole one-wire

La communication one-wire du DHT11 peut être décomposée en trois phases : initiation par l'hôte, réponse du capteur, et transmission des données bit par bit.

(1) **Initiation par l'hôte** : le MCU tire la ligne de données au niveau bas pendant au moins 18 ms, puis la relâche et attend la réponse du DHT11. Cette action sert à réveiller le capteur en mode veille, l'informant qu'une transmission de données est imminente.

(2) **Réponse du capteur** : après avoir détecté l'impulsion d'initiation, le DHT11 sort d'abord un niveau bas de 80 μs , puis un niveau haut de 80 μs , informant ainsi l'hôte de sa disponibilité. L'hôte confirme la présence du capteur en capturant cette séquence d'impulsions.

(3) **Transmission de la trame de données** : le DHT11 rapporte la température et l'humidité sous forme de trame de 40 bits. Chaque bit commence par un niveau bas de 50 μs , suivi d'une largeur de niveau haut déterminant la valeur logique : 26–28 μs représente un "0", et environ 70 μs représente un "1". Les 40 bits sont successivement : 8 bits de partie entière de l'humidité, 8 bits de partie décimale de l'humidité, 8 bits de partie entière de la température, 8 bits de partie décimale de la température, et 8 bits de somme de contrôle.

Le pilote VirtualDHT11 reproduit complètement la séquence temporelle du protocole décrit ci-dessus ; l'implémentation clé est la suivante :

```
1 class VirtualDHT11:
2     def __init__(self, gpio, pin):
3         self.gpio = gpio
4         self.pin = pin
5         self.humidity = 50.0      # Humidite par default 50 %
6         self.temperature = 25.0   # Temperature par default 25 °C
7
```

```
8  def set_environment(self, env):
9      self.humidity = env.get('humidity', 50.0)
10     self.temperature = env.get('temperature', 25.0)
11
12  def read(self):
13      # Simulation du signal de demarrage
14      self.gpio.set_mode(self.pin, 'OUTPUT')
15      self.gpio.write(self.pin, 0)
16      delay_ms(20)
17      self.gpio.write(self.pin, 1)
18      delay_us(30)
19      self.gpio.set_mode(self.pin, 'INPUT')
20
21      # Simulation du signal de reponse et transmission des donnees
22      # Retourne (humidite, temperature)
23      h_int = int(self.humidity)
24      h_dec = int((self.humidity - h_int) * 10)
25      t_int = int(self.temperature)
26      t_dec = int((self.temperature - t_int) * 10)
27      checksum = (h_int + h_dec + t_int + t_dec) & 0xFF
28
29      data = [h_int, h_dec, t_int, t_dec, checksum]
30      return self.humidity + random.gauss(0, 1.5), \
31             self.temperature + random.gauss(0, 0.8)
```

Listing 5: Signal de demarrage et lecture de donnees du DHT11

Dans le Code 5, le VirtualDHT11 reproduit la séquence temporelle du protocole one-wire en inversant les niveaux du GPIO. Lors de la génération des données, les paramètres environnementaux sont encapsulés dans une trame de 40 bits avec une somme de contrôle, puis du bruit gaussien est injecté selon l'Équation 5 : l'écart-type du canal d'humidité est fixé à 1,5 % RH, et celui du canal de température à 0,8 °C, correspondant à la classe de précision du DHT11 réel.

4.3 Capteur de qualite de l'air MQ-135

4.3.1 Principe du capteur

Le MQ-135 présente une haute sensibilité à l'ammoniac, aux sulfures, aux vapeurs de benzène et à la fumée. Sa couche sensible utilise un film semi-conducteur SnO₂ ; les molécules de gaz cibles réagissent à sa surface selon des réactions d'oxydo-réduction, provoquant une variation de la résistance volumique, qui reflète à son tour la concentration

de gaz.

La resistance de l'element sensible R_S du MQ-135 et la concentration de gaz C sont liees par une relation en loi de puissance :

$$\frac{R_S}{R_0} = A \cdot C^{-B} \quad (6)$$

où R_0 est la valeur de resistance de reference du capteur dans l'air pur, A et B sont des constantes empiriques liees au type de gaz, et C est la concentration de gaz (en ppm). Pour les scenarios generaux de surveillance de la qualite de l'air, on prend generalement $A \approx 116,6$ et $B \approx 2,77$.

4.3.2 Deduction tension-concentration

Le module MQ-135 utilise en interne un circuit diviseur de tension pour convertir la variation de resistance en signal de tension. La tension de sortie et la resistance de l'element sensible satisfont :

$$V_{out} = V_{cc} \cdot \frac{R_L}{R_S + R_L} \quad (7)$$

où V_{cc} est la tension d'alimentation (generalement 5 V), et R_L est la resistance de charge (potentiometre ajustable).

On en deduit la resistance de l'element sensible :

$$R_S = R_L \cdot \left(\frac{V_{cc}}{V_{out}} - 1 \right) \quad (8)$$

En reintroduisant cette expression dans l'Equation 6, on obtient l'expression analytique de la concentration de gaz :

$$C = \left(\frac{A \cdot R_0}{R_S} \right)^{\frac{1}{B}} = \left(\frac{A \cdot R_0 \cdot V_{out}}{R_L \cdot (V_{cc} - V_{out})} \right)^{\frac{1}{B}} \quad (9)$$

4.3.3 Implementation du pilote virtuel

Le pilote `VirtualMQ135` prend le parametre environnemental `air_quality` (en ppm) comme entree, et deduit la tension ADC selon les Equations 6 et 9. De plus, le pilote reproduit trois comportements typiques du dispositif reel :

(1) **Simulation de prechauffage** : le capteur MQ-135 necessite un certain temps de prechauffage apres la mise sous tension pour atteindre un etat de fonctionnement stable, le temps de prechauffage typique etant de 20 secondes. Le pilote virtuel enregistre l'heure de demarrage apres l'initialisation, et retourne des donnees invalides ou de faible precision pendant la periode de prechauffage.

(2) **Reponse non lineaire** : selon la relation en loi de puissance, la concentration de gaz et la valeur de resistance presentent une relation non lineaire. Dans la zone de faible

concentration, la sensibilité du capteur est élevée ; dans la zone de forte concentration, la sensibilité diminue progressivement. Le pilote virtuel reproduit complètement cette caractéristique non linéaire.

(3) **Compensation de température et d'humidité** : la sortie du MQ-135 est influencée par la température et l'humidité ambiantes. Le pilote virtuel introduit des coefficients simples de compensation de température et d'humidité, corrigeant la valeur de concentration en fonction de la température et de l'humidité actuelles, afin d'améliorer le réalisme des données simulées.

4.4 Capteur de luminosité BH1750

4.4.1 Vue d'ensemble du capteur

Le BH1750 délivre directement une quantité numérique d'éclairement en lux via le bus I2C, éliminant le besoin d'un ADC externe et d'un étalonnage fastidieux. Sa large gamme de mesure, sa haute précision et sa simplicité de câblage en font un choix très répandu dans les scénarios d'éclairage automatique, de réglage de rétroéclairage et de détection environnementale.

Le mode de mesure du BH1750 est déterminé par un octet de commande ; le Tableau 5 résume les commandes courantes.

Table 5: Commandes I2C courantes du BH1750

Valeur de commande	Nom	Description
0x01	Power On	Commande de mise sous tension, réveille le capteur
0x00	Power Down	Commande de mise hors tension, passe en mode basse consommation
0x10	Continuous H-Res Mode	Mode haute résolution continu (résolution 1 lux, temps de mesure 120 ms)
0x11	Continuous H-Res2 Mode	Mode haute résolution 2 continu (résolution 0,5 lux, temps de mesure 120 ms)
0x13	Continuous L-Res Mode	Mode basse résolution continu (résolution 4 lux, temps de mesure 16 ms)
0x20	One Time H-Res Mode	Mode haute résolution unique

4.4.2 Formule de calcul de la luminosité

En mode H-Res, le BH1750 délivre un comptage brut sur 16 bits (raw value), qui est multiplié par un facteur de sensibilité de 1,2 lux/comptage pour obtenir l'éclairement réel

:

$$E_v = \text{raw} \times 1,2 \quad (10)$$

où E_v est l'éclairement (en lux), et raw est les données brutes 16 bits retournées par le capteur.

Si la valeur raw est 500, l'éclairement correspondant est $500 \times 1,2 = 600$ lux.

Le **VirtualBH1750**, après réception de la commande via I2C, calcule le comptage brut en fonction du paramètre environnemental actuel **illuminance**, selon l'Equation 10 en sens inverse :

$$\text{raw} = \left\lfloor \frac{E_v}{1,2} \right\rfloor + N(0, \sigma^2) \quad (11)$$

où σ prend 3 unités de comptage, correspondant à un bruit de mesure d'environ 3,6 lux, en accord avec les spécifications de la fiche technique du BH1750.

4.5 Capteur de pression barométrique BMP280

4.5.1 Vue d'ensemble du capteur

Le BMP280, produit par Bosch, est basé sur une technologie MEMS, combinant petit volume, faible consommation et haute précision. On le retrouve dans les stations météorologiques, les altimètres, la navigation intérieure et le maintien d'altitude des drones. Le Tableau 6 énumère ses principaux indicateurs techniques.

Table 6: Principaux paramètres techniques du BMP280

Paramètre	Spécification
Plage de mesure de la pression	300 hPa – 1100 hPa
Précision de mesure de la pression	$\pm 0,12$ hPa (valeur typique)
Plage de mesure de la température	$-40^\circ\text{C} - +85^\circ\text{C}$
Précision de mesure de la température	$\pm 0,5^\circ\text{C}$ (à 25°C)
Précision absolue	± 1 hPa
Interface de communication	I2C ou SPI
Identifiant de la puce	0x58

4.5.2 Vérification de l'identifiant de la puce

Le BMP280 possède un registre d'identifiant de puce en lecture seule à l'adresse 0xD0, dont la valeur après réinitialisation est 0x58. Après la mise sous tension, l'hôte lit généralement ce registre pour confirmer la présence du dispositif. Le **VirtualBMP280** intègre ce comportement dès la phase d'initialisation, et une lecture de l'adresse 0xD0 retourne toujours 0x58.

4.5.3 ADC 24 bits et coefficients de calibration

Le BMP280 integre un ADC 24 bits quantifiant respectivement les sorties analogiques des capteurs de pression et de temperature. Les canaux de pression et de temperature ont tous deux une resolution effective de 20 bits, emballes dans des registres de 3 octets. Cette resolution extremement elevee lui permet de capturer des fluctuations de pression au niveau du millibar.

Le BMP280 reel stocke les coefficients de compensation dans la NVM (adresses 0x88–0xA1) lors de l'etalonnage en usine. L'hote doit extraire ces coefficients avant la lecture, puis executer dans le MCU un algorithme de compensation thermique comprenant des termes quadratiques et cubiques, afin de restituer la grandeur physique a partir de la valeur brute ADC.

La plateforme virtuelle simplifie raisonnablement le processus de compensation, approximant par une relation lineaire le mapping entre la pression, la temperature et les valeurs brutes ADC :

$$\text{raw_P} = \left\lfloor \frac{P}{1013,25} \times 415148 \right\rfloor + N(0, \sigma_P^2) \quad (12)$$

$$\text{raw_T} = \lfloor (T + 40) \times 100 \rfloor + N(0, \sigma_T^2) \quad (13)$$

où P est la valeur de pression ambiante (en hPa), T est la valeur de temperature ambiante (en °C), 415148 est la valeur brute ADC typique correspondant a 1013,25 hPa (pression atmospherique standard), et σ_P et σ_T sont les ecarts-types du bruit.

Le developpement de la formule 12 repose sur un fait empirique : en mode de sur-echantillonnage $\times 16$, la valeur brute ADC typique correspondant a la pression atmospherique standard de 1013,25 hPa est d'environ 415148. La relation de proportionnalite pression–valeur brute ainsi etablie omet les corrections complexes de compensation thermique et de non-linearite presentes dans la puce reelle, mais elle conserve une precision suffisante pour les demonstrations pedagogiques et les verifications courantes.

4.6 Capteur de bruit

4.6.1 Principe du capteur

Le capteur de son (communement le module KY-038 ou base sur le LM393) mesure l'intensite du bruit ambiant, presentee en decibels (dB). Ce type de module est generalement constitue d'un microphone a electret haute sensibilite et d'un circuit de conditionnement de signal, convertissant l'onde de pression acoustique en tension analogique.

Le decibel est une unite de rapport logarithmique. Le niveau de pression acoustique

SPL (Sound Pressure Level) est defini par la formule suivante :

$$L_p = 20 \cdot \log_{10} \left(\frac{p}{p_0} \right) \quad (14)$$

où L_p est le niveau de pression acoustique (en dB), p est la pression acoustique réelle (en Pa), et p_0 est la pression de reference (generalement 20×10^{-6} Pa, c'est-a-dire le seuil d'audition de l'oreille humaine a 1 kHz).

4.6.2 Mapping lineaire tension

Apres amplification et redressement, le module de capteur delivre une tension analogique proportionnelle a la pression acoustique. Dans le modele virtuel, on suppose en outre une relation lineaire entre la tension et le niveau de pression acoustique :

$$V_{out} = V_{offset} + k \cdot L_p \quad (15)$$

où V_{offset} est la tension de decalage (correspondant a la sortie a 0 dB, generalement une petite valeur positive dans la pratique), et k est le coefficient de sensibilite (en V/dB).

Le pilote **VirtualNoise** lit le parametre environnemental **noise_level** (en dB), calcule la tension de sortie selon l'Equation 15, puis la quantifie en valeur numerique via l'ADC. Pour se rapprocher de la realite, la lecture finale est superposee d'un bruit gaussien d'ecart-type 1,5 dB, comparable a la precision typique des sonometres de niveau de consommation.

Le **VirtualNoise** reproduit en outre les trois caracteristiques physiques suivantes :

(1) **Directivite** : un microphone reel possede une certaine directivite, avec une sensibilite differente selon la direction d'arrivee du son. Le capteur virtuel introduit un coefficient de directivite simple pour simuler cette caracteristique physique.

(2) **Reponse en frequence** : l'oreille humaine a une sensibilite differente selon la frequence du son ; la ponderation A (A-weighting) est la methode de ponderation la plus courante pour les niveaux de pression acoustique. Le capteur virtuel delivre par default un niveau de pression acoustique pondere A, plus proche de la perception subjective de l'oreille humaine.

(3) **Maintien du pic** : certains capteurs de bruit supportent une fonction de detection de pic, enregistrant le niveau de pression acoustique maximal sur une periode donnee. Le capteur virtuel implemente un mode de maintien du pic optionnel, enregistrant et retournant la valeur maximale en dB sur une fenetre de temps specifiee.

4.7 Conception de la boucle d'acquisition de donnees

La boucle d'acquisition de donnees pilote le flux principal de la station de surveillance virtuelle : a intervalles fixes, elle reveille chaque capteur pour accomplir l'échantillonnage, puis pousse les resultats dans la chaine de stockage et de presentation. Un cycle unique est decompose en six maillons :

1. **Lecture des parametres environnementaux** : lire les reglages actuels des parametres environnementaux depuis le fichier partage `env_control.txt`, incluant la temperature, l'humidite, la pression, la luminosite, la qualite de l'air et le niveau sonore.
2. **Mise a jour de l'environnement des capteurs** : transmettre les parametres environnementaux lus aux instances de capteurs virtuels, afin que les capteurs puissent generer des donnees simulees en fonction des derniers reglages environnementaux.
3. **Execution de la lecture des capteurs** : appeler successivement la methode `read()` de chaque capteur pour obtenir les valeurs de mesure actuelles. L'ordre de lecture est generalement : DHT11 (temperature/humidite) → BMP280 (pression/temperature) → BH1750 (luminosite) → MQ-135 (qualite de l'air) → capteur de bruit.
4. **Formatage des donnees** : convertir les lectures brutes de chaque capteur en unites standard (lux, hPa, ppm, dB, etc.), et organiser les donnees selon une structure de donnees predefinie sous forme de dictionnaire ou de format JSON.
5. **Stockage des donnees** : inserer l'enregistrement formate dans la table `sensor_data` de la base de donnees SQLite, tout en mettant a jour le cache des dernieres donnees en memoire.
6. **Verification des alarmes** : parcourir la configuration des alarmes de chaque capteur, verifier si la lecture actuelle depasse la plage de seuils. En cas de depassement, enregistrer le journal d'alarme et mettre a jour l'etat d'alarme.

Afin d'éviter qu'un cycle d'échantillonnage trop long ne provoque une gigue de periode, le systeme introduit une strategie de compensation de periode. Soit T_{sample} (en secondes) la periode cible, et $t_{elapsed}$ le temps reel ecole pour ce cycle ; le temps de sommeil est alors calcule selon la formule :

$$t_{sleep} = \max(0, T_{sample} - t_{elapsed}) \quad (16)$$

L'Equation 16 garantit que la periode réelle n'est pas inferieure a la valeur definie. Si un cycle d'échantillonnage a deja depasse la duree prevue, la phase de sommeil est saute et on passe directement au cycle suivant, en priorisant l'actualite des donnees.

```
1 import time
2
3 SAMPLE_PERIOD = 2.0  # Periode d'echantillonnage de 2 secondes
4
5 def data_collection_loop(sensors, db, alarm_config):
6     """Boucle principale d'acquisition de donnees"""
7     while True:
8         t_start = time.time()
9
10        # Etape 1 : lecture des parametres environnementaux
11        env = read_environment_control()
12
13        # Etape 2 : mise a jour de l'environnement des capteurs
14        for sensor in sensors:
15            sensor.set_environment(env)
16
17        # Etape 3 : execution de la lecture des capteurs
18        readings = {}
19        for sensor in sensors:
20            readings.update(sensor.read())
21
22        # Etape 4 : formatage des donnees
23        record = format_sensor_data(readings)
24
25        # Etape 5 : stockage des donnees
26        db.insert_sensor_data(record)
27
28        # Etape 6 : verification des alarmes
29        check_alarms(record, alarm_config, db)
30
31        # Compensation de periode
32        t_elapsed = time.time() - t_start
33        t_sleep = max(0, SAMPLE_PERIOD - t_elapsed)
34        time.sleep(t_sleep)
```

Listing 6: Implementation principale de la boucle d'acquisition de donnees

Le Code 6 presente l'implementation complete de `data_collection_loop`. La constante `SAMPLE_PERIOD` definit la periode d'echantillonnage, et la logique de compensation de periode supprime la derive temporelle. Le dictionnaire `readings` transporte les resultats intermediaires entre les etapes, et l'enregistrement formate `record` est finalement

stocke en base de données et participe à la détermination des alarmes.

5 Conception et réalisation du logiciel

La plateforme matérielle virtuelle STM32 et les pilotes de capteurs constituent la base d'exécution sous-jacente. Au-dessus, quatre unités centrales – le module de stockage de données, le système d'alarmes, le système de visualisation Web et le panneau de contrôle – collaborent pour assurer respectivement la persistance des données, la détection d'anomalies, la présentation des résultats et la régulation à distance, formant ainsi l'application logicielle complète de la station intelligente de surveillance environnementale.

5.1 Module de stockage des données

Le système de surveillance doit d'abord écrire les lectures des capteurs, l'état de fonctionnement et les événements d'alarmes dans un support de persistance avant de pouvoir effectuer des analyses retrospectives, des analyses statistiques et des audits de journaux. SQLite sert de moteur de base de données de ce système, ne nécessitant pas de processus serveur indépendant ; l'ensemble de la base de données est stockée dans un fichier unique, ce qui facilite considérablement le déploiement et la migration.

5.1.1 Structure des tables de la base de données

La base de données contient trois tables principales : **sensor_data** (table des données des capteurs), **alarm_log** (table des enregistrements d'alarmes) et **system_log** (table des journaux système).

(1) **Table sensor_data** : utilisée pour stocker les données d'acquisition en temps réel des capteurs, c'est la table de plus grande taille. Sa structure est présentée dans le Tableau 7, comportant 10 champs.

Table 7: Structure de la table `sensor_data`

Nom du champ	Type de donnees	Contrainte	Description
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT	Cle primaire auto-incrementee
<code>timestamp</code>	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	Horodatage d'acquisition
<code>temperature</code>	REAL		Valeur de temperature, unite °C
<code>humidity</code>	REAL		Valeur d'humidite, unite % RH
<code>pressure</code>	REAL		Valeur de pression, unite hPa
<code>illuminance</code>	REAL		Eclairement, unite lux
<code>air_quality</code>	REAL		Indice de qualite de l'air, unite ppm
<code>noise_level</code>	REAL		Niveau sonore, unite dB
<code>cpu_temp</code>	REAL		Temperature interne du MCU, unite °C
<code>data_source</code>	TEXT	DEFAULT 'simulation'	Identifiant de la source de donnees

Le champ `id` utilise une strategie de cle primaire auto-incrementee, attribuant un identifiant unique a chaque enregistrement insere ; `timestamp` est automatiquement rempli par la base de donnees au moment de l'insertion, fournissant une base temporelle pour les recherches par periode ; les six champs de `temperature` a `noise_level` mappent un a un les six grandeurs physiques environnementales ; `cpu_temp` conserve la lecture simulee du capteur de temperature interne du MCU virtuel ; `data_source` est marque par default comme `'simulation'`, ce champ laissant une interface d'extension pour l'integration future de materiel reel.

(2) **Table `alarm_log`** : trace le cycle de vie complet des alarmes, de leur generation a leur resolution. Les significations des champs sont les suivantes : `id` est la cle primaire ; `timestamp` enregistre le moment de la premiere activation de l'alarme ; `sensor_type` indique le capteur ayant genere l'alarme ; `alarm_type` distingue les alarmes de seuil haut (HIGH) et de seuil bas (LOW) ; `threshold` enregistre la limite definie au moment du declenchement ; `actual_value` conserve la lecture du capteur au moment du declenchement ; `status` indique si l'alarme est actuellement active (ACTIVE) ou resolue (RESOLVED) ; `resolved_at` inscrit l'horodatage de la resolution de l'alarme.

(3) **Table `system_log`** : capture les evenements cles et les sorties de debogage pendant

le fonctionnement du systeme. Les champs specifiques sont : `id` cle primaire ; `timestamp` inscrit l'heure de generation du journal ; `level` classe la gravite selon quatre niveaux : `DEBUG`, `INFO`, `WARNING`, `ERROR` ; `module` identifie le module fonctionnel ayant genere le journal ; `message` contient le texte specifique ; `metadata` stocke des informations structurees supplementaires au format JSON, facilitant l'analyse automatique ulterieure.

5.1.2 Interface Python sqlite3

La bibliotheque standard `sqlite3` de Python encapsule completement l'interface C de SQLite, permettant aux developpeurs d'executer du SQL, de controler les limites de transaction et de lire les jeux de resultats directement, sans avoir besoin d'installer de pilote de base de donnees supplementaire.

Chaque acces a la base de donnees est enveloppe dans un gestionnaire de contexte ; la connexion est automatiquement liberee a la fin de l'operation, eliminant ainsi le risque de fuite de handles. Le `journal_mode` est explicitement defini sur WAL (Write-Ahead Logging), permettant aux operations de lecture et d'ecriture de progresser en parallele ; lorsque le thread d'acquisition et le service Web accedent simultanement a la meme base de donnees, le temps d'attente des verrous est significativement reduit.

5.1.3 Operations CRUD

Le module de stockage de donnees expose un ensemble d'interfaces CRUD couvrant la creation, la lecture, la mise a jour et la suppression :

(1) **Create (Creation)** : `insert_sensor_data` recoit les lectures des capteurs sous forme de dictionnaire et les ecrit dans la table `sensor_data`. La requete utilise une liaison parametree, empechant ainsi l'execution de SQL malveillant via une entree construite de maniere malintentionnee, et bloquant ainsi les attaques par injection.

(2) **Read (Lecture)** : le module fournit des capacites de recherche multidimensionnelles : `get_latest_reading()` retourne le dernier point d'echantillonnage ; `get_readings_by_range(start,end)` recupere tous les enregistrements dans la fenetre temporelle donnee ; `get_readings_by_sensor(sensor_type,limit)` remonte les N derniers enregistrements historiques par type de capteur.

(3) **Update (Mise a jour)** : `update_alarm_status` marque les alarmes en etat `ACTIVE` comme `RESOLVED`, et definit `resolved_at` sur l'heure systeme actuelle, completant ainsi la boucle de gestion des alarmes.

(4) **Delete (Suppression)** : `delete_old_readings(before_date)` efface les enregistrements expires en fonction d'une limite de date, pouvant etre declenche automatiquement par une tache planifiee ou manuellement par un administrateur, evitant ainsi la croissance continue des donnees historiques et la pression sur le stockage.

5.1.4 Requetes statistiques

En plus des operations CRUD de base, le module integre egalement des capacites d'analyse aggregative, permettant d'extraire des statistiques sur les donnees d'echantillonnage sur une periode donnee. Les dimensions d'agregation courantes sont les suivantes :

(1) **Requete de moyenne** (AVG) : calcule la moyenne arithmetique des lectures du capteur dans la fenetre de temps selectionnee, caracterisant le niveau global du parametre environnemental sur cette periode.

(2) **Requete de minimum** (MIN) : extrait la valeur d'echantillonnage la plus basse sur cette periode, aidant a detecter des valeurs anormalement basses ou des conditions extremes.

(3) **Requete de maximum** (MAX) : extrait la valeur d'echantillonnage la plus haute sur cette periode, aidant a detecter des valeurs anormalement hautes ou des conditions extremes.

(4) **Requete d'ecart-type** (STDEV) : mesure le degre de dispersion des points d'echantillonnage autour de la moyenne ; une valeur plus elevee indique des fluctuations environnementales plus intenses.

Le modele SQL de requete statistique est le suivant :

```
1 SELECT
2     AVG(temperature) as avg_temp,
3     MIN(temperature) as min_temp,
4     MAX(temperature) as max_temp,
5     AVG(humidity) as avg_humidity,
6     MIN(pressure) as min_pressure,
7     MAX(pressure) as max_pressure
8 FROM sensor_data
9 WHERE timestamp BETWEEN ? AND ?;
```

Listing 7: Modele SQL de requete statistique

Le modele SQL ci-dessus utilise des fonctions d'agregation pour resumer en une seule fois les indicateurs statistiques de plusieurs capteurs. Les limites temporelles dans la clause WHERE sont transmises sous forme de parametres, preservant a la fois la flexibilite de la requete et evitant les risques d'injection lies a la concatenation de chaines.

5.2 Conception du systeme d'alarmes

Le systeme d'alarmes balaie en continu les lectures en temps reel de chaque capteur ; des qu'un depassement de seuil est detecte, il genere une notification pour rappeler au personnel d'exploitation d'intervenir rapidement et de traiter l'anomalie environnementale.

5.2.1 Configuration des seuils d'alarme

Le dictionnaire `ALARM_CONFIG` maintient des limites superieures et inferieures independantes pour chaque capteur : `high_threshold` definit le seuil haut, et `low_threshold` definit le seuil bas. Lorsque la lecture depasse le seuil haut, une alarme de type HIGH est generee ; lorsqu'elle tombe en dessous du seuil bas, une alarme de type LOW est generee ; les deux cas ne se chevauchent pas.

L'exemple de configuration d'alarme par default est le suivant :

```
1 ALARM_CONFIG = {
2     'temperature': {
3         'high_threshold': 35.0,      # Seuil d'alarme haute temperature
4         35 °C
5         'low_threshold': 10.0,      # Seuil d'alarme basse temperature
6         10 °C
7         'unit': 'C',
8         'name': 'Temperature'
9     },
10    'humidity': {
11        'high_threshold': 80.0,      # Seuil d'alarme haute humidite 80
12        %
13        'low_threshold': 20.0,      # Seuil d'alarme basse humidite 20
14        %
15        'unit': '% RH',
16        'name': 'Humidite'
17    },
18    'pressure': {
19        'high_threshold': 1050.0,    # Seuil d'alarme haute pression
20        1050 hPa
21        'low_threshold': 980.0,      # Seuil d'alarme basse pression
22        980 hPa
23        'unit': 'hPa',
24        'name': 'Pression'
25    },
26    'air_quality': {
27        'high_threshold': 500.0,    # Seuil d'alarme qualite de l'air
28        500 ppm
29        'low_threshold': None,      # Pas de seuil bas
30        'unit': 'ppm',
31        'name': 'Qualite de l'air'
32    },
33    # ... autres configurations de capteurs
```

Listing 8: Exemple de configuration des seuils d'alarme

Comme le montre le Code 8, les éléments de configuration de chaque capteur incluent la valeur du seuil, l'unité, le nom en français, etc. Lorsque le `low_threshold` d'un capteur est défini sur `None`, cela signifie que l'alarme de valeur basse n'est pas surveillée, et seule l'alarme de valeur haute est monitorée.

5.2.2 Logique de vérification des alarmes

La fonction `check_all` pilote l'ensemble du processus de détermination des alarmes : elle lit successivement la configuration de chaque capteur dans `ALARM_CONFIG`, compare la valeur d'échantillonnage actuelle avec l'intervalle de seuils, et déclenche ou efface l'alarme en fonction du résultat de la comparaison. Les étapes spécifiques sont les suivantes :

(1) **Vérification itérative** : extraire un par un les éléments de capteur de `ALARM_CONFIG`, et comparer la valeur d'échantillonnage actuelle avec l'intervalle de seuils défini pour ce capteur.

(2) **Détermination du seuil haut** : lorsque la valeur d'échantillonnage est supérieure au `high_threshold`, le système recherche si une alarme HIGH de ce capteur est déjà en état ACTIVE : si c'est le cas, il rafraîchit la `actual_value` ; sinon, il insère un nouvel enregistrement d'alarme ACTIVE.

(3) **Détermination du seuil bas** : lorsque la valeur d'échantillonnage est inférieure au `low_threshold`, le processus de déclenchement de l'alarme de type LOW suit une logique symétrique à celle du seuil haut.

(4) **Recupération de l'alarme** : lorsque la valeur d'échantillonnage retombe dans l'intervalle de seuils, le système marque en masse toutes les alarmes ACTIVE de ce capteur comme RESOLVED, et inscrit l'horodatage `resolved_at`.

(5) **Persistance** : que ce soit pour une nouvelle alarme ou une transition d'état, l'interface de base de données est immédiatement appelée pour persister les données, garantissant ainsi la traçabilité de l'ensemble de la chaîne d'alarmes.

```
1 def check_all(readings, config, db):
2     """
3     Verifie l'etat d'alarme de tous les capteurs
4     Parametres :
5         readings : dictionnaire des lectures actuelles des capteurs
6         config :   dictionnaire de configuration des alarmes
7         db :       instance d'operation de base de donnees
8     """
9     for sensor, cfg in config.items():
```

```
10     value = readings.get(sensor)
11     if value is None:
12         continue
13
14     # Verification du seuil haut
15     high = cfg.get('high_threshold')
16     if high is not None and value > high:
17         trigger_alarm(db, sensor, 'HIGH', high, value)
18
19     # Verification du seuil bas
20     low = cfg.get('low_threshold')
21     if low is not None and value < low:
22         trigger_alarm(db, sensor, 'LOW', low, value)
23
24     # Verification de recuperation
25     if (high is None or value <= high) and \
26         (low is None or value >= low):
27         resolve_alarm(db, sensor)
```

Listing 9: Logique centrale de verification des alarmes

Comme le montre le Code 9, la fonction `check_all` implemente le processus complet de verification des alarmes. `trigger_alarm` et `resolve_alarm` sont des fonctions auxiliaires, chargees respectivement de creer une nouvelle alarme et de recuperer une alarme existante.

5.2.3 Persistance des enregistrements d'alarme

L'enregistrement d'alarme conserve le contexte complet au moment du declenchement, incluant le type de capteur, la limite de seuil et la valeur d'echantillonnage reelle, fournissant des donnees brutes pour l'analyse retrospective et l'audit. Son modele de donnees est decrit dans le Tableau 8.

Table 8: Structure de la table alarm_log

Nom du champ	Type de donnees	Contrainte	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Cle primaire auto-incrementee
timestamp	DATETIME	NOT NULL	Heure de declenchement de l'alarme
sensor_type	TEXT	NOT NULL	Type de capteur
alarm_type	TEXT	NOT NULL	Type d'alarme : HIGH/LOW
threshold	REAL	NOT NULL	Seuil de declenchement
actual_value	REAL	NOT NULL	Valeur reellev au moment du declenchement
status	TEXT	DEFAULT 'ACTIVE'	Etat : ACTIVE/RESOLVED
resolved_at	DATETIME		Heure de resolution

5.3 Systeme de visualisation Web

La couche de visualisation Web fait le pont entre l'utilisateur et la station de surveillance virtuelle : les donnees des capteurs sous-jacents sont rendues de maniere intuitive via des graphiques et des tableaux de bord, tandis que l'utilisateur peut ajuster les parametres environnementaux via le panneau de controle et consulter les notifications d'anomalies dans le panneau d'alarmes.

5.3.1 Conception des routes Flask

Flask assume la responsabilite du service Web backend, exposant un ensemble d'API RESTful pour consommation par le frontend. La methode HTTP, le chemin et la fonction de chaque point de terminaison sont resumes dans le Tableau 9.

Table 9: Conception des routes API RESTful Flask

Methode	Chemin	Fonction	Description
GET	/	Page d'accueil	Retourne la page de visualisation Web
GET	/api/sensors/latest	Donnees les plus recentes	Retourne les dernieres lectures de tous les capteurs
GET	/api/sensors/history	Donnees historiques	Retourne les donnees historiques dans la plage de temps specifiee
GET	/api/alarms/active	Alarmes actives	Retourne la liste des alarmes non resolues actuelles
GET	/api/alarms/history	Historique des alarmes	Retourne tous les enregistrements d'alarmes (supporte la pagination)
POST	/api/control	Controle environnemental	Recoit les instructions de reglage des parametres environnementaux
GET	/api/stats	Statistiques	Retourne les informations statistiques pour la periode specifiee

5.3.2 Format de reponse JSON RESTful

Chaque API retourne un corps JSON contenant systematiquement trois cles de premier niveau : **status**, **data** et **message**. **status** prend la valeur 'ok' ou 'error', refletant le resultat du traitement de la requete ; **data** porte la charge utile réelle au niveau metier ; **message** est rempli avec une description d'erreur lisible uniquement en cas de scenario anormal.

Exemple de reponse reussie :

```

1 {
2   "status": "ok",
3   "data": {
4     "temperature": 26.5,
5     "humidity": 58.2,
6     "pressure": 1013.8,
7     "illuminance": 320.0,
8     "air_quality": 120.5,
9     "noise_level": 45.3,
10    "timestamp": "2024-01-15T09:30:00"
11  },
12  "message": null
13 }
```

Listing 10: Exemple de reponse API reussie

5.3.3 Configuration du graphique a deux courbes Chart.js

Le frontend utilise Chart.js pour rendre les courbes de tendance en temps reel. Afin d'afficher simultanement sur une seule toile les evolutions de deux capteurs associes, le systeme adopte un graphique lineaire a deux axes Y (Dual Line Chart). Le code d'initialisation Chart.js pour la tendance conjointe temperature/humidite est presente ci-dessous :

```
1 const ctx = document.getElementById('tempHumChart').getContext('2d');
2 const tempHumChart = new Chart(ctx, {
3     type: 'line',
4     data: {
5         labels: [],          // Tableau des etiquettes temporelles
6         datasets: [
7             {
8                 label: 'Temperature (°C)',
9                 data: [],
10                borderColor: 'rgb(255, 99, 132)',
11                backgroundColor: 'rgba(255, 99, 132, 0.1)',
12                yAxisID: 'y-temp',
13                tension: 0.3
14            },
15            {
16                label: 'Humidite (% RH)',
17                data: [],
18                borderColor: 'rgb(54, 162, 235)',
19                backgroundColor: 'rgba(54, 162, 235, 0.1)',
20                yAxisID: 'y-humidity',
21                tension: 0.3
22            }
23        ]
24    },
25    options: {
26        responsive: true,
27        interaction: { mode: 'index', intersect: false },
28        scales: {
29            'y-temp': {
30                type: 'linear',
```

```
31         display: true,
32         position: 'left',
33         title: { display: true, text: 'Temperature (°C)' }
34     },
35     'y-humidity': {
36         type: 'linear',
37         display: true,
38         position: 'right',
39         title: { display: true, text: 'Humidite (% RH)' }
40     }
41 }
42 }
43 });
```

Listing 11: Configuration du graphique a deux courbes Chart.js

Le graphique a deux courbes utilise `yAxisID` pour mapper la courbe de temperature sur l'axe Y gauche et la courbe d'humidite sur l'axe Y droit, eliminant ainsi le probleme de compression visuelle cause par les differences d'unites de mesure. Le parametre `tension` est fixe a 0,3, preservant a la fois les caracteristiques des points d'inflexion des donnees et donnant aux lignes brisees un aspect global plus lisse et naturel.

5.3.4 Mecanisme d'interrogation a 1 seconde

Le frontend interroge le backend a intervalles de 1000 millisecondes via `setInterval`, tirant activement les dernieres lectures des capteurs et l'etat des alarmes, realisant ainsi un rafraichissement quasi-temps-reel de l'interface.

```
1 // Interrogation toutes les 1000 ms (1 seconde)
2 setInterval(() => {
3     fetch('/api/sensors/latest')
4         .then(r => r.json())
5         .then(data => {
6             if (data.status === 'ok') {
7                 updateSensorCards(data.data);
8                 updateCharts(data.data);
9             }
10        })
11        .catch(err => console.error('Polling error:', err));
12
13    fetch('/api/alarmes/active')
14        .then(r => r.json())
```

```
15         .then(data => {
16             if (data.status === 'ok') {
17                 updateAlarmPanel(data.data);
18             }
19         });
20     }, 1000);
```

Listing 12: Implementation du mecanisme d'interrogation du frontend

Le frontend interroge en parallele les deux points de terminaison `/api/sensors/latest` et `/api/alarms/active`, garantissant que les valeurs des cartes, les courbes de tendance et la liste des alarmes sont toujours alignees avec l'etat backend. Par rapport a la poussée WebSocket, l'interrogation fait un leger compromis sur la reactivite, mais elle est legere a implementer et largement compatible avec les navigateurs, et elle est tout a fait adequate pour les scenarios de rafraichissement a l'echelle de la seconde. Des iterations ulterieures pourraient migrer vers SSE ou WebSocket pour reduire davantage les surcoûts de connexion.

5.4 Conception du panneau de controle

L'utilisateur modifie directement les parametres environnementaux virtuels dans le navigateur via le panneau de controle, observant en temps reel les variations des lectures des capteurs dans des conditions specifiques. Cette capacite est particulierement cruciale pour les demonstrations en classe et le depannage.

5.4.1 Conception du mecanisme IPC

La boucle d'acquisition du MCU virtuel reside dans un thread Python independant, tandis que le service Web Flask s'execute dans le thread principal ; les deux doivent echanger des instructions de controle via un canal inter-threads. Le systeme choisit une solution IPC basee sur le partage de fichiers : `env_control.txt` sert de medium d'echange de donnees bidirectionnel.

Le choix de l'IPC par fichier repose sur quatre considerations : premierement, il n'est pas necessaire d'introduire de dependances supplementaires telles que Redis ou ZeroMQ, reduisant ainsi la complexite du systeme ; deuxiemement, le contenu du fichier est preserve apres le redemarrage du processus, offrant une capacite intrinseque de persistance d'etat ; troisiemement, le format texte permet aux developpeurs de consulter et de modifier facilement le contenu manuellement, reduisant les couts de debogage ; quatriemement, la lecture et l'ecriture de fichiers sont des capacites fondamentales de la grande majorite des langages de programmation, minimisant les obstacles a la migration lors de l'introduction future de modules heterogenes.

Le fichier `env_control.txt` adopte un format simple de paires cle-valeur, avec un parametre par ligne, au format `cle=valeur` :

```
1 temperature=25.0
2 humidity=60.0
3 pressure=1013.25
4 illuminance=500.0
5 air_quality=100.0
6 noise_level=40.0
7 wind_speed=5.0
```

Listing 13: Exemple de format du fichier de controle environnemental

Le fichier `env_control.txt` maintient sept parametres environnementaux. Le thread d'acquisition lit ce fichier au debut de chaque cycle d'échantillonnage, rafraichissant l'entree environnementale des capteurs virtuels avec les valeurs les plus recentes ; le backend Web recrit les clees correspondantes apres reception d'une requete POST, et les nouveaux parametres prennent effet des le prochain cycle d'acquisition.

5.4.2 Analyse de la commande set

Le point de terminaison `/api/control` recoit un corps de requete POST au format JSON, extrait les parametres environnementaux a modifier, puis les reecrit dans `env_control.txt`. La logique d'analyse suit une semantique de mise a jour partielle : autant de clees que presentes dans le corps de la requete sont mises a jour, tandis que les autres clees conservent leur valeur d'origine, evitant ainsi a l'utilisateur d'avoir a soumettre l'integralite des parametres a chaque fois.

```
1 @app.route('/api/control', methods=['POST'])
2 def set_environment():
3     """Regle les parametres environnementaux"""
4     data = request.get_json()
5     if not data:
6         return jsonify({'status': 'error', 'message': 'No data'}),
7             400
8
9     # Lecture des parametres environnementaux actuels
10    env = read_env_file()
11
12    # Mise a jour des parametres specifiques
13    valid_keys = ['temperature', 'humidity', 'pressure',
14                  'illuminance', 'air_quality', 'noise_level',
15                  'wind_speed']
```

```

15     for key in valid_keys:
16         if key in data:
17             env[key] = float(data[key])
18
19     # Ecriture dans le fichier
20     write_env_file(env)
21
22     return jsonify({'status': 'ok', 'data': env})

```

Listing 14: Interface de reglage des parametres environnementaux

Le chemin d'exécution de `set_environment` se divise en trois etapes : charger le fichier environnemental existant, ecraser les entrees correspondantes selon les paires cle-valeur de la requete, puis reecrire le resultat sur le disque. La liste blanche `valid_keys` combinee a la conversion forcee en `float` bloque les clees illegales et les entrees de type errone.

5.4.3 Limites des plages de parametres

La simulation des capteurs manque de tolerance physique aux entrees extremes ; le backend definit donc une plage de validite pour chacun des sept parametres environnementaux. Lorsque l'utilisateur soumet une valeur hors limites, l'API retourne un code d'etat 400 avec un message d'erreur, demandant une correction avant reessai. Les limites superieures et inferieures specifiques ainsi que les valeurs par defaut de chaque parametre sont presentees dans le Tableau 10.

Table 10: Plages valides des parametres environnementaux

Nom du parametre	Min	Max	Valeur par defaut	Unite
temperature	-40.0	+85.0	25.0	°C
humidity	0.0	100.0	50.0	% RH
pressure	300.0	1100.0	1013.25	hPa
illuminance	0.0	65535.0	500.0	lux
air_quality	0.0	10000.0	100.0	ppm
noise_level	0.0	150.0	40.0	dB
wind_speed	0.0	100.0	5.0	m/s

La conception des plages de chaque parametre est strictement alignee sur les fiches techniques des capteurs reels et les lois physiques : les limites de temperature sont coherentes avec la plage de fonctionnement nominale du BMP280 ; l'humidite est limitee

a 0–100 %, car une valeur hors de cet intervalle n'a pas de sens physique ; la plage de pression couvre les scenarios extremes allant de la basse pression en haute altitude a la haute pression au niveau de la mer ; la limite superieure de 65535 lux pour l'eclairment correspond exactement a la pleine echelle du BH1750 en mode H-Res. Ces frontieres interceptent la grande majorite des entrees erronees, maintenant ainsi la rationalite des sorties simulees.

6 Tests et validation du systeme

Afin de verifier si les modules de la plateforme materielle virtuelle STM32, des pilotes de capteurs, du stockage de donnees, du systeme d'alarmes et de la visualisation Web atteignent les indicateurs attendus, il est necessaire de mettre en place un systeme de tests systematique et d'executer une validation multidimensionnelle. Les sections suivantes presentent respectivement l'environnement de test, la couverture fonctionnelle, les benchmarks de performance, la precision des capteurs et la fiabilite des alarmes.

6.1 Environnement de test

La configuration de l'environnement de test de ce systeme est presentee dans le Tableau 11.

Table 11: Configuration de l'environnement de test du systeme

Element	Configuration
Systeme d'exploitation	Windows 11 Professionnel 64 bits
Processeur	Intel Core i7-12700H @ 2,3 GHz
Memoire	16 Go DDR5
Version Python	Python 3.11.6
Version Flask	Flask 3.0.0
Version SQLite	SQLite 3.42.0
Navigateur	Google Chrome 120.0

La machine de test est un ordinateur portable equipe d'un Intel Core i7-12700H et de 16 Go de memoire DDR5, une configuration sensiblement equivalente aux environnements de developpement actuels. L'ensemble du processus de test est execute en boucle fermee en local, eliminant ainsi les interferences liees a la latence du reseau et a la charge des serveurs distants.

6.2 Tests fonctionnels

Les tests fonctionnels verifient le comportement du systeme element par element par rapport au cahier des charges. La conception des cas de test couvre six modules : l'initialisation du MCU virtuel, la lecture des donnees des capteurs, le chargement de la page Web, la recuperation des donnees via API, le declenchement des seuils d'alarme, et l'emission des parametres environnementaux via le panneau de controle. Les resultats des tests sont resumes dans le Tableau 12.

Table 12: Resultats des tests fonctionnels

ID	Element de test	Etapes de test	Res
FT-01	Demarrage MCU	Initialiser l'instance VirtualSTM32	Systeme d
FT-02	Activation ADC	Configurer l'horloge et le bit d'activation de l'ADC	ADC p
FT-03	Lecture DHT11	Appeler la methode dht11.read()	Retourne les valeurs
FT-04	Lecture MQ-135	Attendre le prechauffage puis appeler read()	Retourne la valeur
FT-05	Lecture BH1750	Envoyer la commande de mesure puis lire	Retourne la va
FT-06	Lecture BMP280	Lire l'ID de la puce et obtenir les donnees	Retourne la p
FT-07	Lecture bruit	Appeler la methode noise.read()	Retour
FT-08	Acces page d'accueil Web	Acceder via le navigateur a http://localhost:5000/	Page se c
FT-09	Recuperation donnees API	Requete GET /api/sensors/latest	Retourne
FT-10	Declenchement alarme haute temperature	Regler temperature=40	Enregistre
FT-11	Declenchement alarme basse pression	Regler pressure=920	Enregistre
FT-12	Operation panneau de controle	Modifier les parametres environnementaux via l'interface Web	Fichier de

Les 12 cas de test du Tableau 12 ont tous ete acceptes. La couche des pilotes de capteurs (FT-03 a FT-07) confirme que les cinq capteurs virtuels peuvent retourner correctement les grandeurs physiques ; la couche des services Web (FT-08, FT-09) confirme que le rendu de page et les interfaces JSON repondent normalement ; la couche de la

logique d'alarmes (FT-10, FT-11) confirme que les dépassements de seuils haut et bas sont tous deux captures ; la couche de controle (FT-12) confirme que le fichier de parametres environnementaux peut etre modifie a distance.

6.3 Tests de performance

Les tests de performance se concentrent sur la latence de reponse et l'occupation des ressources, verifiant si le systeme peut toujours repondre aux exigences de reactivite de la surveillance a l'echelle de la seconde en cas de fonctionnement continu.

6.3.1 Temps de reponse des API

En utilisant la bibliotheque `requests` de Python, 100 requetes consecutives ont ete envoyees aux differents points de terminaison du backend Flask, enregistrant le temps aller-retour de chaque requete et calculant les valeurs statistiques. Les resultats de mesure sont presentes dans le Tableau 13.

Table 13: Resultats des tests de temps de reponse des API

Point de terminaison API	Temps de reponse moyen	Min	Max
GET /api/sensors/latest	12,5 ms	8,2 ms	28,4 ms
GET /api/sensors/history	45,3 ms	32,1 ms	112,6 ms
GET /api/alarms/active	9,8 ms	6,5 ms	22,1 ms
POST /api/control	15,2 ms	10,3 ms	35,7 ms
GET /api/stats	38,6 ms	25,4 ms	98,3 ms

Le temps de reponse moyen de tous les points de terminaison est controle en dessous de 100 millisecondes, conforme a la ligne de base de performance predefinie. `/api/sensors/latest` et `/api/alarms/active` ne consultent qu'un seul enregistrement le plus recent, avec un temps moyen d'environ 10 ms ; `/api/sensors/history` et `/api/stats` necessitent le balayage d'une plage temporelle et l'execution d'operations d'agregation, avec un temps moyen croissant jusqu'a 40–45 ms, le surcout supplementaire provenant principalement de la recherche par intervalle et du calcul de groupement de la base de donnees.

6.3.2 Stabillite de la periode d'acquisition

La gigue de la periode d'acquisition se reflète directement dans la fluidite des courbes de tendance et influence egalement la precision temporelle de la determination des alarmes. L'experience a enregistre en continu le temps d'execution reel de 100 cycles, en prenant comme reference la periode cible $T_{sample} = 2,0$ secondes, et en évaluant l'amplitude de la derive.

La moyenne des périodes sur 100 échantillonnages est de 2,003 secondes, avec un écart-type de 0,018 seconde et une dérive positive maximale de +0,052 seconde. La cause fondamentale de la gigue réside dans la fluctuation du temps consommé par chaque écriture SQLite et par la logique de détermination des alarmes ; même lorsque les données historiques s'accumulent à une certaine échelle, le mode WAL maintient le temps d'attente des verrous à un niveau faible. Dans l'ensemble, la stabilité de la période est suffisante pour supporter la surveillance en temps réel, et les courbes de tendance peuvent fidèlement restituer l'évolution des paramètres environnementaux.

6.3.3 Occupation mémoire

En utilisant le module intégré `tracemalloc` de Python et la bibliothèque tierce `psutil`, l'empreinte mémoire du système a été suivie en continu. Après une heure de fonctionnement continu, les détails d'occupation de chaque module sont présentés dans le Tableau 14.

Table 14: Résultats des tests d'occupation mémoire

Module	Occupation mémoire
Service Web Flask	Environ 45 Mo
Instance MCU virtuelle et capteurs	Environ 12 Mo
Thread d'acquisition de données	Environ 8 Mo
Cache base de données SQLite	Environ 5 Mo
Ressources page frontend	Environ 3 Mo (cote navigateur)
Total	Environ 73 Mo

La consommation mémoire globale du système est d'environ 73 Mo, dont le service Web Flask occupe environ 45 Mo, cette partie étant principalement générée par le runtime de l'interpréteur Python et la structure propre du framework Flask. 73 Mo constituent une charge extrêmement légère dans les environnements informatiques modernes, et le système peut être déployé sans problème même sur des périphériques de bord tels que le Raspberry Pi.

6.4 Vérification de la précision des capteurs

La crédibilité de la station de surveillance virtuelle dépend dans une large mesure de la précision de simulation des capteurs. Afin de quantifier cet indicateur, les performances d'erreur de chaque capteur virtuel sont comparées horizontalement avec la précision nominale indiquée dans les fiches techniques officielles.

6.4.1 Analyse comparative de la precision

Les ecarts de precision entre les capteurs virtuels et les puces reelles sont resumes dans le Tableau 15.

Table 15: Comparaison de precision entre capteurs virtuels et reels

Capteur	Parametre mesure	Precision reelle	Precision simulee	Ec
DHT11	Temperature	$\pm 2^{\circ}\text{C}$	$\pm 0,8^{\circ}\text{C}$	Meilleure
DHT11	Humidite	$\pm 5\% \text{ RH}$	$\pm 1,5\% \text{ RH}$	Meilleure
MQ-135	Concentration de gaz	$\pm 15\%$	$\pm 8\%$	Meilleure
BH1750	Eclairement	$\pm 20\%$	$\pm 5\%$	Meilleure
BMP280	Pression	$\pm 1 \text{ hPa}$	$\pm 0,5 \text{ hPa}$	Meilleure
BMP280	Temperature	$\pm 0,5^{\circ}\text{C}$	$\pm 0,3^{\circ}\text{C}$	Meilleure
Capteur de bruit	Niveau de pression acoustique	$\pm 3 \text{ dB}$	$\pm 1,5 \text{ dB}$	Meilleure

Le Tableau 15 montre que les intervalles d'erreur de tous les capteurs virtuels sont plus etroits que la precision nominale des puces reelles. La simulation logicielle elimine les sources de perturbation physiques telles que la derive thermique, le vieillissement des composants, le couplage electromagnetique et le bruit d'alimentation ; le bruit de sortie n'est injecte que par le generateur de nombres aleatoires gaussiens, offrant ainsi une coherence et une repetabilite significativement superieures au materiel reel.

Une precision de simulation superieure a la precision reelle n'affaiblit pas la valeur pedagogique et de validation de la plateforme. Dans un environnement de classe, un bruit plus faible permet aux etudiants d'identifier plus facilement la relation de correspondance entre la reponse du capteur et la variable environnementale, evitant d'etre masques par les erreurs aleatoires du materiel reel. Si une approximation plus proche des conditions reelles est necessaire, il suffit d'augmenter l'ecart-type σ du bruit gaussien pour ramener le capteur virtuel a un niveau d'erreur proche de celui de la puce reelle, permettant ainsi des tests plus realistes.

6.4.2 Tests de stabilite a long terme

Le capteur virtuel BMP280 a ete soumis a un test de stabilite continu de 24 heures : les parametres environnementaux etaient verrouilles a $P = 1013,25 \text{ hPa}$ et $T = 25,0^{\circ}\text{C}$, avec une periode d'echantillonnage de 2 secondes. Les resultats statistiques sur 43 200 points de donnees montrent une pression moyenne de $1013,28 \text{ hPa}$, un ecart-type de $0,42 \text{ hPa}$, une deviation maximale positive de $+1,8 \text{ hPa}$ et une deviation maximale negative de $-1,5 \text{ hPa}$. La distribution des points d'echantillonnage suit une loi normale, sans derive monotone ni decalage systematique, prouvant ainsi la stabilite numerique du capteur virtuel lors d'un fonctionnement prolonge.

6.5 Validation de la fonction d'alarme

La fiabilité du module d'alarmes détermine si l'utilisateur peut être informé et agir dès la dégradation de l'environnement. Deux scénarios typiques sont conçus ci-dessous pour valider l'ensemble du processus de déclenchement, d'affichage et de récupération des alarmes.

6.5.1 Scénario d'alarme haute température

Scénario : la température ambiante est élevée à 40 °C via le panneau de contrôle, valeur qui dépasse le seuil haut de température de 35 °C.

Étapes de test :

1. Régler le paramètre température à 40,0 via le panneau de contrôle.
2. Attendre que la boucle d'acquisition de données exécute au moins un cycle (environ 2 secondes).
3. Interroger la table `alarm_log` pour vérifier si un nouvel enregistrement d'alarme de température a été généré.
4. Accéder au panneau d'alarmes de l'interface Web pour confirmer que l'information d'alarme s'affiche correctement.
5. Ramener la température à 25,0 et attendre la récupération de l'alarme.

Observation : après avoir réglé la température à 40,0, le système a réussi à identifier le dépassement de seuil au deuxième cycle d'échantillonnage. Un nouvel enregistrement avec `status='ACTIVE'` et `alarm_type='HIGH'` a été ajouté à `alarm_log`, avec une `actual_value` de 40,1 °C (bruit simulé superposé) et un `threshold` de 35,0 °C. Le panneau d'alarmes Web a mis en évidence cette anomalie avec une carte rouge. Après retour de la température à la plage normale, le même enregistrement a automatiquement migré vers l'état RESOLVED environ 2 secondes plus tard, avec l'horodatage de résolution inscrit dans le champ `resolved_at`.

6.5.2 Scénario d'alarme basse pression

Scénario : la pression atmosphérique ambiante est abaissée à 920 hPa via le panneau de contrôle, valeur qui tombe en dessous du seuil bas de pression de 980 hPa.

Étapes de test :

1. Régler le paramètre pression à 920,0 via le panneau de contrôle.
2. Attendre que la boucle d'acquisition de données exécute au moins un cycle.

3. Interroger la table `alarm_log` pour verifier si un nouvel enregistrement d'alarme de pression a ete genere.
4. Acceder au panneau d'alarmes de l'interface Web pour confirmer que l'information d'alarme s'affiche correctement.
5. Ramener la pression a 1013,25 et attendre la recuperation de l'alarme.

Observation : apres avoir regle la pression a 920,0, le systeme a correctement identifie le depassement du seuil bas, generant un enregistrement d'alarme avec `alarm_type='LOW'` et une `actual_value` de 919,6 hPa. L'interface Web a distingue l'alarme basse avec une etiquette orange (les alarmes hautes utilisent le rouge). Apres retour de la pression a la normale, cette alarme a ete automatiquement marquee comme RESOLVED au cycle d'echantillonnage suivant.

6.5.3 Scenario d'alarmes simultanees multiples

Validation du scenario concurrent : elever simultanement la temperature a 42,0 °C et la qualite de l'air a 800 ppm, forçant la generation d'alarmes HIGH a la fois sur les dimensions temperature et qualite de l'air. Le systeme a genere des enregistrements ACTIVE independants pour les deux capteurs, sans ecrasement mutuel au niveau de la base de donnees ; le panneau d'alarmes Web a affiche en parallele les deux anomalies, avec des couleurs correspondant a chaque capteur ; lors de leur recuperation ulterieure, les transitions d'etat se sont egalement effectuees de maniere isolee. Ce scenario prouve que le module d'alarmes conserve une logique correcte et une stabilite de fonctionnement dans des conditions de concurrence multi-sources.

6.6 Presentation de l'interface Web

L'architecture de l'information et la disposition visuelle de l'interface Web determinant directement l'efficacite avec laquelle l'utilisateur obtient l'etat de l'environnement. Le frontend de ce systeme est divise en six zones fonctionnelles :

(1) **Cartes de donnees des capteurs** : six cartes de donnees sont disposees horizontalement en haut de l'ecran, chaque carte indiquant en haut le nom du capteur, affichant au centre la valeur actuelle et l'unite en caracteres agrandis, et accompagnant en bas une fleche de tendance (↑ hausse, ↓ baisse, → stable). Lorsque la lecture se situe dans la plage de securite, le fond de la carte est vert ; lorsqu'elle approche du seuil, il passe progressivement au jaune, formant ainsi un avertissement visuel intuitif.

(2) **Graphique de tendance temperature/humidite** : dans la partie gauche du centre, un graphique lineaire a deux axes Y est place, avec la serie rouge suivant l'evolution de la temperature et la serie bleue suivant l'evolution de l'humidite, l'axe horizontal representant le temps d'echantillonnage. Le survol de la souris sur un point de donnees affiche

une info-bulle avec la valeur précise, et la mise à jour de la courbe est accompagnée d'une animation de transition fluide.

(3) **Graphique qualité de l'air et éclairement** : dans la partie droite du centre, un autre groupe de graphiques linéaires à deux axes Y est placé de manière symétrique, avec la courbe violette représentant la qualité de l'air et la courbe jaune représentant l'éclairage. Étant donné l'écart significatif entre les ordres de grandeur du ppm et du lux, des échelles indépendantes à gauche et à droite évitent que la série de faible amplitude ne soit comprimée en une ligne droite.

(4) **Panneau d'informations statistiques** : dans la partie inférieure gauche, des indicateurs agrégés sur trois granularités temporelles – 1 heure, 24 heures et 7 jours – sont résumés, couvrant les dimensions de la moyenne et des valeurs extrêmes. Les données sont tirées dynamiquement via `/api/stats`, et l'utilisateur peut changer la fenêtre statistique via un menu déroulant.

(5) **Panneau d'alarmes** : dans la partie inférieure droite, divisé en deux sections supérieure et inférieure. La section supérieure liste les alarmes en état ACTIVE en temps réel, avec un fond rouge/orange attirant l'attention ; chaque alarme indique le type, le capteur, l'heure de déclenchement et la valeur de dépassement. La section inférieure affiche les enregistrements historiques dans l'ordre chronologique inverse, avec support de la pagination.

(6) **Panneau de contrôle** : la barre latérale droite intègre sept groupes de contrôles de réglage, correspondant respectivement à la température, l'humidité, la pression, l'éclairage, la qualité de l'air, le bruit et la vitesse du vent. Chaque contrôle est accompagné de la lecture actuelle et de la plage valide ; les entrées hors limites sont immédiatement interceptées et un message d'avertissement convivial est affiché.

Le frontend adopte une mise en page responsive en grille, reorganisant automatiquement les cartes et les graphiques en fonction de la largeur de la fenêtre, préservant ainsi la lisibilité tant sur ordinateur de bureau que sur mobile. La palette de couleurs utilise un fond gris clair, un thème principal bleu-vert, et du rouge-orange pour les alertes, avec une progression hiérarchique qui aide l'utilisateur à localiser les informations clés en 3 secondes.

7 Conclusion et perspectives

7.1 Synthèse des travaux

Cette recherche prend la virtualisation de la station intelligente de surveillance environnementale STM32 comme objet central, et réalise un travail complet allant de la théorie au prototype. Les productions principales peuvent être synthétisées en trois points :

(1) **Construction d'une plateforme matérielle virtuelle STM32 haute fi-**

delite. En prenant le STM32F103C8T6 comme reference, une plateforme materielle MCU virtuelle complete a ete realisee en Python, couvrant le noyau Cortex-M3, la cartographie memoire en trois segments (Flash/SRAM/registres), le timer SysTick, les GPIO, l'ADC, l'I2C, l'UART et les TIM. Cette plateforme ne depend d'aucun materiel physique et peut executer des programmes embarques directement sur Windows, Linux et macOS. Les experiences de validation montrent que le comportement de lecture et d'ecriture des registres virtuels est hautement coherent avec les descriptions du manuel de reference du STM32 reel, suffisant pour supporter l'enseignement en classe et la validation de prototypes precoces.

(2) **Etablissement d'un cadre de virtualisation unifie des capteurs environnementaux.** Sur la base du materiel virtuel, la migration des pilotes de cinq capteurs environnementaux courants – DHT11, MQ-135, BH1750, BMP280 et capteur de bruit – a ete realisee. Au niveau du cadre, le mapping des parametres environnementaux, la conversion des grandeurs physiques et le mecanisme d'injection de bruit gaussien $Y = X + N(0, \sigma^2)$ ont ete unifies, de sorte que la sortie simulee respecte a la fois les lois physiques et conserve les caracteristiques aleatoires reelles. Chaque pilote reproduit integralement les protocoles de communication one-wire et I2C, les sequences de mise sous tension et les chaines de conditionnement de signal, avec une erreur globale superieure a la precision nominale des fiches techniques officielles.

(3) **Realisation d'un systeme complet de visualisation Web et de controle des donnees.** Le backend, base sur Flask, fournit des API RESTful ; SQLite assure le stockage persistant des donnees des capteurs, des enregistrements d'alarmes et des journaux systeme ; le frontend, en HTML/CSS/JavaScript natif combine avec Chart.js, realise les graphiques lineaires en temps reel, la retrospection historique, les popups d'alarmes et le controle par curseurs des parametres environnementaux. Le panneau de controle et le MCU virtuel communiquent en boucle fermee via le fichier partage `env_control.txt` ; l'utilisateur peut deplacer un curseur dans le frontend et observer le changement des lectures des capteurs au cycle d'echantillonnage suivant.

La phase de mesure a confirme : la plateforme virtuelle peut fonctionner de maniere stable 7j/7 et 24h/24, la derive de la periode d'acquisition est controlee a moins de $\pm 0,052$ seconde, le temps de reponse moyen des API Web est inferieur a 100 millisecondes, et la consommation memoire totale du site est d'environ 73 Mo. Le module d'alarmes capture et archive avec precision les depassements de seuils haut et bas, et l'interface frontend presente une hierarchie d'information claire. Cette solution purement logicielle reduit le cout d'accès des experiences embarquees a un niveau extremement bas, tout en fournissant un bac a sable de validation a iteration rapide et a consommation materielle nulle pour les projets IoT en phase initiale.

7.2 Perspectives de travaux futurs

La version actuelle de la station de surveillance environnementale virtuelle couvre déjà les scénarios centraux d'enseignement et de validation de prototypes, mais cinq directions méritent une exploration approfondie en termes de profondeur technique et d'intégration écologique :

(1) **Simulation du système d'exploitation en temps réel FreeRTOS.** La plateforme actuelle ne supporte que des programmes bare-metal, et l'ordonnancement des tâches repose entièrement sur l'interrogation et les interruptions. L'étape suivante pourrait consister à porter la simulation de l'ordonnanceur FreeRTOS, supportant la préemption par priorité, le tourniquet (time slicing), la synchronisation par sémaphores, la communication par files de messages et l'allocation dynamique de mémoire, permettant aux apprenants d'expérimenter la programmation multitâche dans un environnement virtuel et de réduire la distance cognitive entre la salle de classe et le terrain industriel.

(2) **Support du protocole MQTT.** L'interrogation HTTP est acceptable pour les scénarios à l'échelle de la seconde, mais les surcoûts de connexion et le délai de transmission limitent son extensibilité dans les scénarios IoT à haute concurrence. Après l'introduction de MQTT, le MCU virtuel publie les données des capteurs vers un Broker en tant qu'éditeur, tandis que le frontend Web et les terminaux mobiles reçoivent les mises à jour de sujet en tant qu'abonnés, réalisant une véritable transmission à la demande. En tant que protocole de facto dans le domaine de l'IoT, l'ajout de MQTT améliorera directement la capacité d'interfacage de la plateforme avec les systèmes industriels réels.

(3) **Extension à davantage de types de capteurs.** La bibliothèque de capteurs actuelle couvre cinq dimensions – température, humidité, pression, éclairage et qualité de l'air. Des modules supplémentaires pourraient être connectés par la suite, tels que le module de dioxyde de carbone MH-Z19C, le module de particules PMS5003, le module ultraviolet GUVA-S12SD, le capteur d'humidité du sol et le pluviomètre à auget basculeur. Avec l'élargissement des types, la plateforme pourrait s'intégrer de manière transparente dans des domaines verticaux supplémentaires tels que les serres agricoles intelligentes, la surveillance environnementale maillée urbaine et la santé et sécurité professionnelle en usine.

(4) **Simulation des modes basse consommation.** Le STM32F103C8T6 intègre trois niveaux d'état basse consommation : Sleep, Stop et Standby. L'autonomie dans les scénarios alimentés par batterie dépend fortement de la gestion fine de ces états. Les travaux futurs pourraient simuler dans la plateforme virtuelle le processus complet de contrôle d'horloge, de coupure d'alimentation des périphériques et de mapping des sources de réveil, permettant aux apprenants de comprendre les compromis entre consommation d'énergie et latence de réponse sans avoir besoin de consommer de véritables batteries.

(5) **Développement d'une application mobile.** L'interface Web actuelle possède

déjà une capacité multiplateforme, mais reste limitée dans les aspects de l'écran tactile de petite taille, de la conservation en arrière-plan et des notifications locales par les contraintes du bac à sable du navigateur. Le développement d'une application complémentaire basée sur Flutter ou React Native permettrait d'accéder aux notifications système, à la diffusion vocale des alarmes anormales, et de lier les données de surveillance à la localisation GPS, réalisant ainsi une perception environnementale véritablement portable.

Du point de vue pédagogique, les moyens de virtualisation éliminent les contraintes rigides de l'achat de matériel et de la réservation de laboratoires ; les étudiants peuvent reproduire à tout moment l'ensemble du processus de développement embarqué sur leur ordinateur personnel. Du point de vue de l'ingénierie, un environnement de vérification purement logiciel permet d'itérer rapidement sur les solutions architecturales à un coût extrêmement faible en phase initiale de projet. À mesure que la demande industrielle en talents embarqués continue de croître, ce type de plateforme virtuelle haute fidélité jouera un rôle de plus en plus crucial dans la réduction du cycle de formation des talents et la diminution des coûts d'essai-erreur de l'innovation.

References

Références

- [1] Liu Huoliang, Yang Sen. Guide pratique du developpement de bibliotheques STM32 : base sur STM32F4 [M]. Beijing : China Machine Press, 2019.
- [2] Zhengdian Yuanzi. Guide de developpement STM32F1 : version bibliotheque de fonctions [M]. Guangzhou : Zhengdian Yuanzi, 2022.
- [3] Wang Lihua, Zhang Minghui. Conception et realisation d'une plateforme d'experimentation virtuelle pour systemes embarques [J]. Computer Education, 2021(5) : 112-117.
- [4] Chen Zhigang, Li Xiaofeng. Vue d'ensemble de la recherche sur les systemes intelligents de surveillance environnementale bases sur l'IoT [J]. Transducer and Microsystem Technologies, 2020, 39(8) : 1-4.
- [5] Zhou Runjing, Zhang Lina. Simulation STM32 basee sur Proteus et exemples d'application [M]. Beijing : Publishing House of Electronics Industry, 2018.
- [6] ARM Limited. Cortex-M3 Technical Reference Manual [EB/OL]. ARM Developer, 2021.
- [7] STMicroelectronics. RM0008 Reference Manual : STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs [EB/OL]. ST Community, 2021.
- [8] Sensirion AG. SHT30-DIS Datasheet : Humidity and Temperature Sensor [EB/OL]. Sensirion, 2020.
- [9] Bosch Sensortec. BMP280 Digital Pressure Sensor Datasheet [EB/OL]. Bosch, 2020.
- [10] ROHM Semiconductor. BH1750FVI-E Datasheet : Digital 16bit Serial Output Type Ambient Light Sensor [EB/OL]. ROHM, 2019.
- [11] Hanwei Electronics. MQ-135 Gas Sensor Datasheet [EB/OL]. Hanwei, 2018.
- [12] Flask Documentation. Flask 3.0 Quick Start [EB/OL]. Pallets Projects, 2023. <https://flask.palletsprojects.com/>
- [13] Mozilla Developer Network. Fetch API Web APIs [EB/OL]. MDN Web Docs, 2023. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [14] Chart.js Documentation. Chart.js 4.x Getting Started [EB/OL]. Chart.js, 2023. <https://www.chartjs.org/docs/>

- [15] Wang Zhihua, Liu Yang. Conception et realisation d'un systeme de surveillance environnementale IoT base sur le protocole MQTT [J]. Internet of Things Technologies, 2022, 12(3) : 45-48.
- [16] Li Jianguo, Zhang Wei. Recherche sur l'application de la technologie de simulation dans l'enseignement des systemes embarques [J]. Journal of Electrical and Electronic Education, 2021, 43(2) : 78-82.
- [17] SQLite Consortium. SQLite Documentation [EB/OL]. SQLite, 2023. <https://www.sqlite.org/docs.html>
- [18] Zhao Minghua, Sun Li. Application de la technologie d'instrumentation virtuelle dans l'enseignement des capteurs [J]. Experimental Technology and Management, 2020, 37(6) : 156-160.
- [19] QEMU Project. QEMU Documentation [EB/OL]. QEMU, 2023. <https://www.qemu.org/documentation/>
- [20] Yang Liqiang, Wang Haitao. Conception d'une plateforme d'enseignement d' experimentation embarquee a distance basee sur le Web [J]. Computer Engineering and Applications, 2019, 55(14) : 245-250.